

Item Type	Item Code	Length Octet	Contents of Value
Routers	3	N	IP addresses of N/4 routers
Time Server	4	N	IP addresses of N/4 time servers
IEN116 Server	5	N	IP addresses of N/4 IEN116 servers
Domain Server	6	N	IP addresses of N/4 DNS servers
Log Server	7	N	IP addresses of N/4 log servers
Quote Server	8	N	IP addresses of N/4 quote servers
Lpr Servers	9	N	IP addresses of N/4 lpr servers
Impress	10	N	IP addresses of N/4 Impress servers
RLP Server	11	N	IP addresses of N/4 RLP servers
Hostname	12	N	N bytes of client host name
Boot Size	13	2	2-octet integer size of boot file
RESERVED	128-254	-	Reserved for site specific use

Figure 23.3 Types and contents of items in the *VENDOR-SPECIFIC AREA* of a BOOTP reply that have variable lengths.

parameters for each host, and then store the information in a BOOTP server configuration file — BOOTP does not include a way to dynamically assign values to individual machines. In particular, a manager must assign each host an IP address, and must configure the server so it understands the mapping from host identifier to IP address.

Static parameter assignment works well if computers remain at fixed locations and a manager has sufficient IP addresses to assign each computer a unique IP address. However, in cases where computers move frequently or the number of physical computers exceeds the number of available IP host addresses, static assignment incurs excessive overhead.

To understand how the number of computers can exceed the number of available IP addresses, consider a LAN in a college laboratory that has been assigned a /24 address that allows up to 254 hosts. Assume that because the laboratory only has seats for 30 students, the college schedules labs at ten different times during the week to accommodate up to 300 students. Further assume that each student carries a personal notebook computer that they use in the lab. At any given time, the net has at most 30 active computers. However, because the network address can accommodate at most 254 hosts, a manager cannot assign a unique address to each computer. Thus, although resources such as physical connections limit the number of simultaneous connections, the number of potential computers that can use the facility is high. Clearly, a system is inadequate if it requires a manager to change the server's configuration file before a new computer can be added to the network and begin to communicate; an automated mechanism is needed.

23.9 Dynamic Host Configuration

To handle automated address assignment, the IETF has designed a new protocol. Known as the *Dynamic Host Configuration Protocol (DHCP)*, the new protocol extends BOOTP in two ways. First, DHCP allows a computer to acquire all the configuration information it needs in a single message. For example, in addition to an IP address, a DHCP message can contain a subnet mask. Second, DHCP allows a computer to obtain an IP address quickly and dynamically. To use DHCP's dynamic address allocation mechanism, a manager must configure a DHCP server by supplying a set of IP addresses. Whenever a new computer connects to the network, the new computer contacts the server and requests an address. The server chooses one of the addresses the manager specified, and allocates that address to the computer.

To be completely general, DHCP allows three types of address assignment; a manager chooses how DHCP will respond for each network or for each host. Like BOOTP, DHCP allows *manual configuration* in which a manager can configure a specific address for a specific computer. DHCP also permits *automatic configuration* in which a manager allows a DHCP server to assign a permanent address when a computer first attaches to the network. Finally, DHCP permits completely *dynamic configuration* in which a server "loans" an address to a computer for a limited time.

Like BOOTP, DHCP uses the identity of the client to decide how to proceed. When a client contacts a DHCP server, the client sends an identifier, usually the client's hardware address. The server uses the client's identifier and the network to which the client has connected to determine how to assign the client and IP address. Thus, a manager has complete control over how addresses are assigned. A server can be configured to allocate addresses to specific computers statically (like BOOTP), while allowing other computers to obtain permanent or temporary addresses dynamically.

23.10 Dynamic IP Address Assignment

Dynamic address assignment is the most significant and novel aspect of DHCP. Unlike the static address assignment used in BOOTP, dynamic address assignment is not a one-to-one mapping, and the server does not need to know the identity of a client *a priori*. In particular, a DHCP server can be configured to permit an arbitrary computer to obtain an IP address and begin communicating. Thus, DHCP makes it possible to design systems that autoconfigure. After such a computer has been attached to a network, the computer uses DHCP to obtain an IP address, and then configures its TCP/IP software to use the address. Of course, autoconfiguration is subject to administrative restrictions — a manager decides whether each DHCP server allows autoconfiguration. To summarize:

Because it allows a host to obtain all the parameters needed for communication without manual intervention, DHCP permits autoconfiguration. Autoconfiguration is, of course, subject to administrative constraints.

To make autoconfiguration possible, a DHCP server begins with a set of IP addresses that the network administrator gives the server to manage. The administrator specifies the rules by which the server operates. A DHCP client negotiates use of an address by exchanging messages with a server. In the exchange, the server provides an address for the client, and the client verifies that it accepts the address. Once a client has accepted an address, it can begin to use that address for communication.

Unlike static address assignment, which permanently allocates each IP address to a specific host, dynamic address assignment is temporary. We say that a DHCP server *leases* an address to a client for a finite period of time. The server specifies the lease period when it allocates the address. During the lease period, the server will not lease the same address to another client. At the end of the lease period, however, the client must renew the lease or stop using the address.

How long should a DHCP lease last? The optimal time for a lease depends on the particular network and the needs of a particular host. For example, to guarantee that addresses can be recycled quickly, computers on a network used by students in a university laboratory might have a short lease period (e.g., one hour). By contrast, a corporate network might use a lease period of one day or one week. To accommodate all possible environments, DHCP does not specify a fixed constant for the lease period. Instead, the protocol allows a client to request a specific lease period, and allows a server to inform the client of the lease period it grants. Thus, a manager can decide how long each server should allocate an address to a client. In the extreme, DHCP reserves a value for *infinity* to permit a lease to last arbitrarily long like the permanent address assignments used in BOOTP.

23.11 Obtaining Multiple Addresses

A multi-homed computer connects to more than one network. When such a computer boots, it may need to obtain configuration information for each of its interfaces. Like a BOOTP message, a DHCP message only provides information about one interface. A computer with multiple interfaces must handle each interface separately. Thus, although we will describe DHCP as if a computer needs only one address, the reader must remember that each interface of a multi-homed computer may be at a different point in the protocol.

Both BOOTP and DHCP use the notion of *relay agent* to permit a computer to contact a server on a nonlocal network. When a relay agent receives a broadcast request from a client, it forwards the request to a server and then returns the reply from the server to the host. Relay agents can complicate multi-homed configuration because a server may receive multiple requests from the same computer. However, although both BOOTP and DHCP use the term *client identifier*, we assume that a multihomed client sends a value that identifies a particular interface (e.g., a unique hardware address). Thus, a server will always be able to distinguish among requests from a multi-homed host, even when the server receives such requests via a relay agent.

23.12 Address Acquisition States

When it uses DHCP to obtain an IP address, a client is in one of six states. The state transition diagram in Figure 23.4 shows events and messages that cause a client to change state.

When a client first boots, it enters the *INITIALIZE* state. To start acquiring an IP address, the client first contacts all DHCP servers in the local net. To do so, the client broadcasts a *DHCPDISCOVER* message and moves to the state labeled *SELECT*. Because the protocol is an extension of BOOTP, the client sends the *DHCPDISCOVER* message in a UDP datagram with the destination port set to the BOOTP port (i.e., port 67). All DHCP servers on the local net receive the message, and those servers that have been programmed to respond to the particular client send a *DHCPOFFER* message. Thus, a client may receive zero or more responses.

While in state *SELECT*, the client collects *DHCPOFFER* responses from DHCP servers. Each offer contains configuration information for the client along with an IP address that the server is offering to lease to the client. The client must choose one of the responses (e.g., the first to arrive), and negotiate with the server for a lease. To do so, the client sends the server a *DHCPREQUEST* message, and enters the *REQUEST* state. To acknowledge receipt of the request and start the lease, the server responds by sending a *DHCPACK*. Arrival of the acknowledgement causes the client to move to the *BOUND* state, where the client proceeds to use the address. To summarize:

To use DHCP, a host becomes a client by broadcasting a message to all servers on the local network. The host then collects offers from servers, selects one of the offers, and verifies acceptance with the server.

23.13 Early Lease Termination

We think of the *BOUND* state as the normal state of operation; a client typically remains in the *BOUND* state while it uses the IP address it has acquired. If a client has secondary storage (e.g., a local disk), the client can store the IP address it was assigned, and request the same address when it restarts again. In some cases, however, a client in the *BOUND* state may discover it no longer needs an IP address. For example, suppose a user attaches a portable computer to a network, uses DHCP to acquire an IP address, and then uses TCP/IP to read electronic mail. The user may not know how long reading mail will require, or the portable computer may allow the server to choose a lease period. In any case, DHCP specifies a minimum lease period of one hour. If after obtaining an IP address, the user discovers that no e-mail messages are waiting to be read, the user may choose to shutdown the portable computer and move to another location.

When it no longer needs a lease, DHCP allows a client to terminate a lease without waiting for the lease to expire. Such termination is helpful in cases where neither the client nor the server can determine an appropriate lease duration at the time the lease is

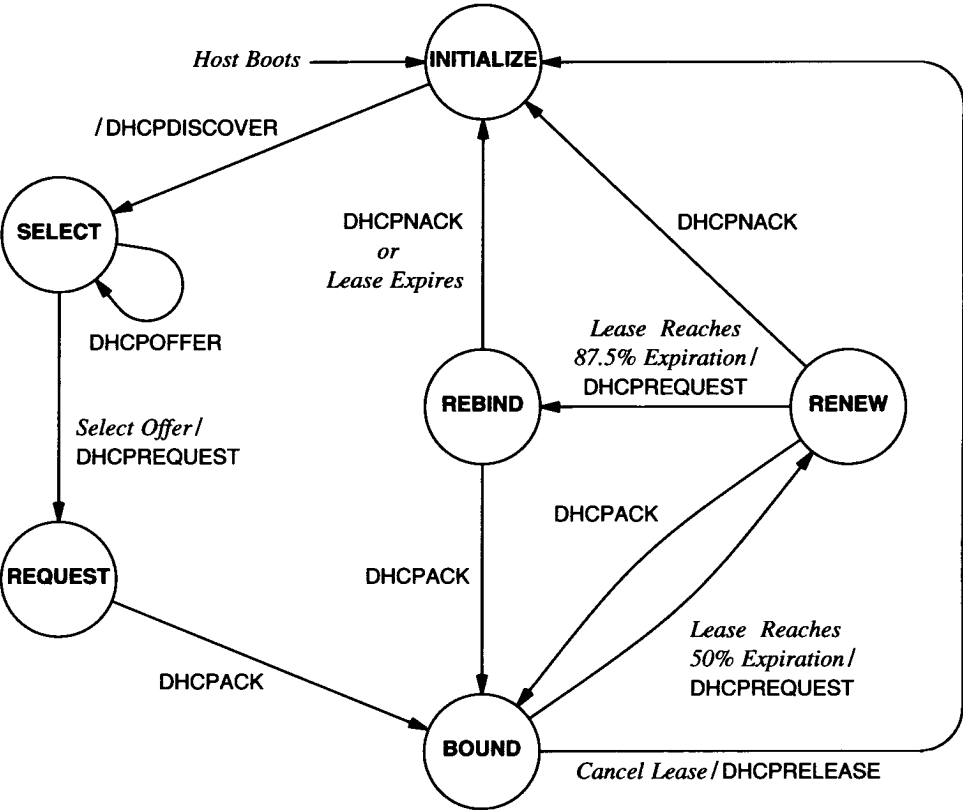


Figure 23.4 The six main states of a DHCP client and transitions among them. Each label on a transition lists the incoming message or event that causes the transmission, followed by a slash and the message the client sends.

granted because it allows a server to choose a reasonably long lease period. Early termination is especially important if the number of IP addresses a server has available is much smaller than the number of computers that attach to the network. If each client terminates its lease as soon as the IP address is no longer needed, the server will be able to assign the address to another client.

To terminate a lease early, a client sends a *DHCPRELEASE* message to the server. Releasing an address is a final action that prevents the client from using the address further. Thus, after transmitting the release message, the client must not send any other datagrams that use the address. In terms of the state transition diagram of Figure 23.4, a host that sends a *DHCPRELEASE* leaves the *BOUND* state, and must start at the *INITIALIZE* state again before it can use IP.

23.14 Lease Renewal States

We said that when it acquires an address, a DHCP client moves to the *BOUND* state. Upon entering the *BOUND* state, the client sets three timers that control lease renewal, rebinding, and expiration. A DHCP server can specify explicit values for the timers when it allocates an address to the client; if the server does not specify timer values, the client uses defaults. The default value for the first timer is one-half of the total lease time. When the first timer expires, the client must attempt to renew its lease. To request a renewal, the client sends a *DHCPREQUEST* message to the server from which the lease was obtained. The client then moves to the *RENEW* state to await a response. The *DHCPREQUEST* contains the IP address the client is currently using, and asks the server to extend the lease on the address. As in the initial lease negotiation, a client can request a period for the extension, but the server ultimately controls the renewal. A server can respond to a client's renewal request in one of two ways: it can instruct the client to stop using the address or it can approve continued use. If it approves, the server sends a *DHCPACK*, which causes the client to return to the *BOUND* state and continue using the address. The *DHCPACK* can also contain new values for the client's timers. If a server disapproves of continued use, the server sends a *DHCPNACK* (negative acknowledgement), which causes the client to stop using the address immediately and return to the *INITIALIZE* state.

After sending a *DHCPREQUEST* message that requests an extension on its lease, a client remains in state *RENEW* awaiting a response. If no response arrives, the server that granted the lease is either down or unreachable. To handle the situation, DHCP relies on a second timer, which was set when the client entered the *BOUND* state. The second timer expires after 87.5% of the lease period, and causes the client to move from state *RENEW* to state *REBIND*. When making the transition, the client assumes the old DHCP server is unavailable, and begins broadcasting a *DHCPREQUEST* message to any server on the local net. Any server configured to provide service to the client can respond positively (i.e., to extend the lease), or negatively (i.e. to deny further use of the IP address). If it receives a positive response, the client returns to the *BOUND* state, and resets the two timers. If it receives a negative response, the client must move to the *INITIALIZE* state, must immediately stop using the IP address, and must acquire a new IP address before it can continue to use IP.

After moving to the *REBIND* state, a client will have asked the original server plus all servers on the local net for a lease extension. In the rare case that a client does not receive a response from any server before its third timer expires, the lease expires. The client must stop using the IP address, must move back to the *INITIALIZE* state, and begin acquiring a new address.

23.15 DHCP Message Format

As Figure 23.5 illustrates, DHCP uses the BOOTP message format, but modifies the contents and meanings of some fields.

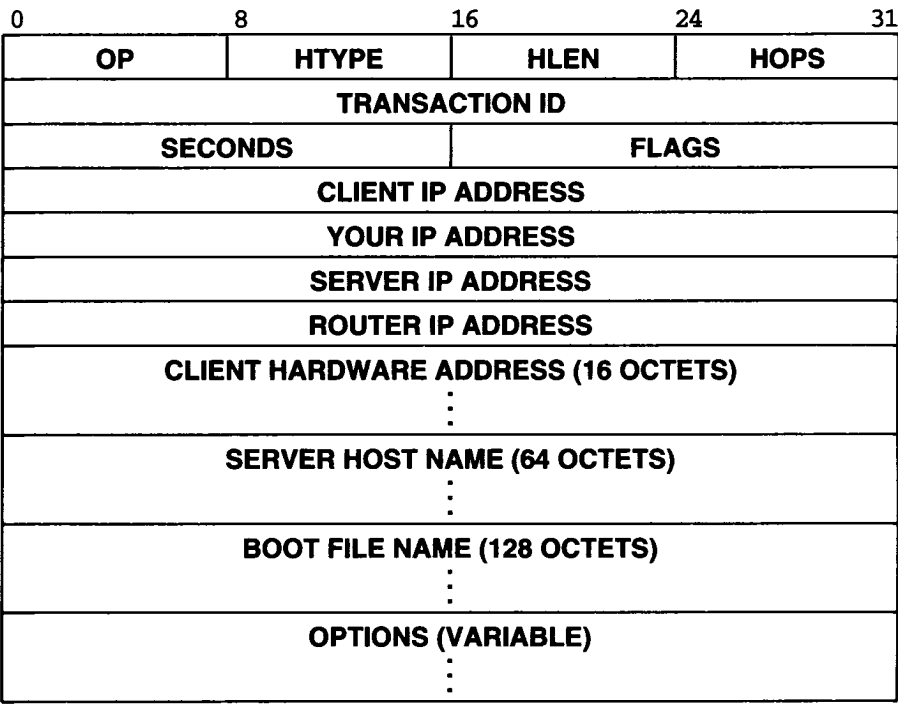


Figure 23.5 The format of a DHCP message, which is an extension of a BOOTP message. The options field is variable length; a client must be prepared to accept at least 312 octets of options.

As the figure shows, most of the fields in a DHCP message are identical to fields in a BOOTP message. In fact, the two protocols are compatible; a DHCP server can be programmed to answer BOOTP requests. However, DHCP changes the meaning of two fields. First, DHCP interprets BOOTP’s *UNUSED* field as a 16-bit *FLAGS* field. In fact, Figure 23.6 shows that only the high-order bit of the *FLAGS* field has been assigned a meaning.

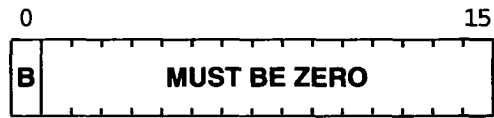


Figure 23.6 The format of the 16-bit *FLAGS* field in a DHCP message. The leftmost bit is interpreted as a broadcast request; all others bits must be set to zero.

Because the DHCP request message contains the client’s hardware address, a DHCP server normally sends its responses to the client using hardware unicast. A client sets the high-order bit in the *FLAGS* field to request that the server respond using hardware broadcast instead of hardware unicast. To understand why a client might choose a broadcast response, recall that while a client communicates with a DHCP server, it does not yet have an IP address. If a datagram arrives via hardware unicast and the destination address does not match the computer’s address, IP can discard the datagram. However, IP is required to accept and handle any datagram sent to the IP broadcast address. To ensure IP software accepts and delivers DHCP messages that arrive before the machine’s IP address has been configured, a DHCP client can request that the server send responses using IP broadcast.

23.16 DHCP Options And Message Type

Surprisingly, DHCP does not add new fixed fields to the BOOTP message format, nor does it change the meaning of most fields. For example, the *OP* field in a DHCP message contains the same values as the *OP* field in a BOOTP message: the message is either a boot request (1) or a boot reply (2). To encode information such as the lease duration, DHCP uses *options*. In particular, Figure 23.7 illustrates the *DHCP message type* option used to specify which DHCP message is being sent.

The options field has the same format as the *VENDOR SPECIFIC AREA*, and DHCP honors all the vendor specific information items defined for BOOTP. As in BOOTP, each option consists of a 1-octet code field and a 1-octet length field followed by octets of data that comprise the option. As the figure shows, the option used to specify a DHCP message type consists of exactly three octets. The first octet contains the code 53, the second contains the length 1, and the third contains a value used to identify one of the possible DHCP messages.

0	8	16	23
CODE (53)	LENGTH (1)	TYPE (1 - 7)	

TYPE FIELD	Corresponding DHCP Message Type
1	DHCPDISCOVER
2	DHCPOFFER
3	DHCPREQUEST
4	DHCPDECLINE
5	DHCPACK
6	DHCPNACK
7	DHCPRELEASE

Figure 23.7 The format of a DHCP message type option used to specify the DHCP message being sent. The table lists possible values of the third octet and their meaning.

23.17 Option Overload

Fields *SERVER HOST NAME* and *BOOT FILE NAME* in the DHCP message header each occupy many octets. If a given message does not contain information in either of those fields, the space is wasted. To allow a DHCP server to use the two fields for other options, DHCP defines an *Option Overload* option. When present, the overload option tells a receiver to ignore the usual meaning of the *SERVER HOST NAME* and *BOOT FILE NAME* fields, and look for options in the fields instead.

23.18 DHCP And Domain Names†

Although it can allocate an IP address to a computer on demand, DHCP does not completely automate all the procedures required to attach a permanent host to an internet. In particular, DHCP does not interact with the domain name system. Thus, the binding between a host name and the IP address DHCP assigns the host must be managed independently.

What name should a host receive when it obtains an IP address from DHCP? Conceptually, there are three possibilities. First, the host does not receive a name. Although it is possible to run client software on a host without a name, using an unnamed computer can be inconvenient. Second, the host is automatically assigned a name along with an IP address. This method is currently popular because names can be preallocated, and no change is required to the DNS. For example, a system administrator can configure the local domain name server to have a host name for each IP address DHCP manages. Once it has been installed in DNS, the name-to-address binding

†Chapter 24 considers the Domain Name System in detail.

remains static. The chief disadvantage of a static binding is that the host receives a new name whenever it receives a new address (e.g., if a host moves from one physical net to another). Third, the host can be assigned a permanent name that remains unchanged. Keeping a permanent host name is convenient because the computer can always be reached via one name, independent of the computer's current location.

Additional mechanisms are needed to support permanent host names. In particular, permanent host names require coordination between DHCP and DNS. A DNS server must change the name-to-address binding whenever a host receives an IP address, and must remove the binding when a lease expires. Although, an IETF working group is currently considering how DHCP should interact with the domain name system, there is currently no protocol for dynamic DNS update. Thus, until a dynamic update mechanism is developed, there is no protocol that maintains permanent host names while allowing DHCP to change IP addresses.

23.19 Summary

The BOOTstrap Protocol, BOOTP, provides an alternative to RARP for a computer that needs to determine its IP address. BOOTP is more general than RARP because it uses UDP, making it possible to extend bootstrapping across a router. BOOTP also allows a machine to determine a router address, a (file) server address, and the name of a program the computer should run. Finally, BOOTP allows administrators to establish a configuration database that maps a generic name, like "unix," into the fully qualified file name that contains a memory image appropriate for the client hardware.

BOOTP is designed to be small and simple enough to reside in a bootstrap ROM. The client uses the limited broadcast address to communicate with the server, and takes responsibility for retransmitting requests if the server does not respond. Retransmission uses an exponential backoff policy similar to Ethernet to avoid congestion.

Designed as a successor to BOOTP, the Dynamic Host Configuration Protocol (DHCP) extends BOOTP in several ways. Most important, DHCP permits a server to allocate IP addresses automatically or dynamically. Dynamic allocation is necessary for environments such as a wireless network where computers can attach and detach quickly. To use DHCP, a computer becomes a client. The computer broadcasts a request for DHCP servers, selects one of the offers it receives, and exchanges messages with the server to obtain a lease on the advertised IP address.

When a client obtains an IP address, the client starts three timers. After the first timer expires, the client attempts to renew its lease. If a second timer expires before renewal completes, the client attempts to rebind its address from any server. If the final timer expires before a lease has been renewed, the client stops using the IP address and returns to the initial state to acquire a new address. A finite state machine explains lease acquisition and renewal.

FOR FURTHER STUDY

BOOTP is a standard protocol in the TCP/IP suite. Further details can be found in Croft and Gilmore [RFC 951], which compares BOOTP to RARP and serves as the official standard. Reynolds [RFC 1084] tells how to interpret the vendor-specific area, and Braden [RFC 1123] recommends using the vendor-specific area to pass the subnet mask.

Droms [RFC 2131] contains the specification for DHCP, including a detailed description of state transitions; another revision is expected soon. A related document, Alexander and Droms [RFC 2132], specifies the encoding of DHCP options and BOOTP vendor extensions. Finally, Droms [RFC 1534] discusses the interoperability of BOOTP and DHCP.

EXERCISES

- 23.1 BOOTP does not contain an explicit field for returning the time of day from the server to the client, but makes it part of the (optional) vendor-specific information. Should the time be included in the required fields? Why or why not?
- 23.2 Argue that separation of configuration and storage of memory images is *not* good. (See RFC 951 for hints.)
- 23.3 The BOOTP message format is inconsistent because it has two fields for client IP address and one for the name of the boot image. If the client leaves its IP address field empty, the server returns the client's IP address in the second field. If the client leaves the boot file name field empty, the server *replaces* it with an explicit name. Why?
- 23.4 Read the standard to find out how clients and servers use the *HOPS* field.
- 23.5 When a BOOTP client receives a reply via hardware broadcast, how does it know whether the reply is intended for another BOOTP client on the same physical net?
- 23.6 When a machine obtains its subnet mask with BOOTP instead of ICMP, it places less load on *other* host computers. Explain.
- 23.7 Read the standard to find out how a DHCP client and server can agree on a lease duration without having synchronized clocks.
- 23.8 Consider a host that has a disk and uses DHCP to obtain an IP address. If the host stores its address on disk along with the date the lease expires, and then reboots within the lease period, can it use the address? Why or why not?
- 23.9 DHCP mandates a minimum address lease of one hour. Can you imagine a situation in which DHCP's minimum lease causes inconvenience? Explain.
- 23.10 Read the RFC to find out how DHCP specifies renewal and rebinding timers. Should a server ever set one without the other? Why or why not?
- 23.11 The state transition diagram does not show retransmission. Read the standard to find out how many times a client should retransmit a request.

- 23.12** Can DHCP guarantee that a client is not “spoofing” (i.e., can DHCP guarantee that it will not send configuration information for host *A* to host *B*)? Does the answer differ for BOOTP? Why or why not?
- 23.13** DHCP specifies that a client must be prepared to handle at least 312 octets of options. How did the number 312 arise?
- 23.14** Can a computer that uses DHCP to obtain an IP address operate a server? If so, how does a client reach the server?

The Domain Name System (DNS)

24.1 Introduction

The protocols described in earlier chapters use 32-bit integers called Internet Protocol addresses (IP addresses) to identify machines. Although such addresses provide a convenient, compact representation for specifying the source and destination in packets sent across an internet, users prefer to assign machines pronounceable, easily remembered names.

This chapter considers a scheme for assigning meaningful high-level names to a large set of machines, and discusses a mechanism that maps between high-level machine names and IP addresses. It considers both the translation from high-level names to IP addresses and the translation from IP addresses to high-level machine names. The naming scheme is interesting for two reasons. First, it has been used to assign machine names throughout the global Internet. Second, because it uses a geographically distributed set of servers to map names to addresses, the implementation of the name mapping mechanism provides a large scale example of the client-server paradigm described in Chapter 21.

24.2 Names For Machines

The earliest computer systems forced users to understand numeric addresses for objects like system tables and peripheral devices. Timesharing systems advanced computing by allowing users to invent meaningful symbolic names for both physical objects (e.g., peripheral devices) and abstract objects (e.g., files). A similar pattern has emerged in computer networking. Early systems supported point-to-point connections between computers and used low-level hardware addresses to specify machines. Internetworking introduced universal addressing as well as protocol software to map universal addresses into low-level hardware addresses. Because most computing environments contain multiple machines, users need meaningful, symbolic names to identify them.

Early machine names reflected the small environment in which they were chosen. It was quite common for a site with a handful of machines to choose names based on the machines' purposes. For example, machines often had names like *research*, *production*, *accounting*, and *development*. Users find such names preferable to cumbersome hardware addresses.

Although the distinction between *address* and *name* is intuitively appealing, it is artificial. Any *name* is merely an identifier that consists of a sequence of characters chosen from a finite alphabet. Names are only useful if the system can efficiently map them to the object they denote. Thus, we think of an IP address as a *low-level name*, and we say that users prefer *high-level names* for machines.

The form of high-level names is important because it determines how names are translated to low-level names or bound to objects, as well as how name assignments are authorized. When only a few machines interconnect, choosing names is easy, and any form will suffice. On the Internet, to which approximately one hundred million machines connect, choosing symbolic names becomes difficult. For example, when its main departmental computer was connected to the Internet in 1980, the Computer Science Department at Purdue University chose the name *purdue* to identify the connected machine. The list of potential conflicts contained only a few dozen names. By mid 1986, the official list of hosts on the Internet contained 3100 officially registered names and 6500 official aliases†. Although the list was growing rapidly in the 1980s, most sites had additional machines (e.g., personal computers) that were not registered.

24.3 Flat Namespace

The original set of machine names used throughout the Internet formed a *flat namespace* in which each name consisted of a sequence of characters without any further structure. In the original scheme, a central site, the Network Information Center (*NIC*), administered the namespace and determined whether a new name was appropriate (i.e., it prohibited obscene names or new names that conflicted with existing names).

The chief advantage of a flat namespace is that names are convenient and short; the chief disadvantage is that a flat namespace cannot generalize to large sets of machines for both technical and administrative reasons. First, because names are drawn from a

†By 1990, more than 137,000 Internet hosts had names, and by 2000 the number exceeded 60 million.

single set of identifiers, the potential for conflict increases as the number of sites increases. Second, because authority for adding new names must rest at a single site, the administrative workload at that central site also increases with the number of sites. To understand the severity of the problem, imagine a rapidly growing internet with thousands of sites, each of which has hundreds of individual personal computers and workstations. Every time someone acquires and connects a new personal computer, its name must be approved by the central authority. Third, because the name-to-address bindings change frequently, the cost of maintaining correct copies of the entire list at each site is high and increases as the number of sites increases. Alternatively, if the name database resides at a single site, network traffic to that site increases with the number of sites.

24.4 Hierarchical Names

How can a naming system accommodate a large, rapidly expanding set of names without requiring a central site to administer it? The answer lies in decentralizing the naming mechanism by delegating authority for parts of the namespace and distributing responsibility for the mapping between names and addresses. TCP/IP internets use such a scheme. Before examining the details of the TCP/IP scheme, we will consider the motivation and intuition behind it.

The partitioning of a namespace must be defined in a way that supports efficient name mapping and guarantees autonomous control of name assignment. Optimizing only for efficient mapping can lead to solutions that retain a flat namespace and reduce traffic by dividing the names among multiple mapping machines. Optimizing only for administrative ease can lead to solutions that make delegation of authority easy but name mapping expensive or complex.

To understand how the namespace should be divided, consider the internal structure of large organizations. At the top, a chief executive has overall responsibility. Because the chief executive cannot oversee everything, the organization may be partitioned into divisions, with an executive in charge of each division. The chief executive grants each division autonomy within specified limits. More to the point, the executive in charge of a particular division can hire or fire employees, assign offices, and delegate authority, without obtaining direct permission from the chief executive.

Besides making it easy to delegate authority, the hierarchy of a large organization introduces autonomous operation. For example, when an office worker needs information like the telephone number of a new employee, he or she begins by asking local clerical workers (who may contact clerical workers in other divisions). The point is that although authority always passes down the corporate hierarchy, information can flow across the hierarchy from one office to another.

24.5 Delegation Of Authority For Names

A hierarchical naming scheme works like the management of a large organization. The namespace is *partitioned* at the top level, and authority for names in subdivisions is passed to designated agents. For example, one might choose to partition the namespace based on *site name* and to delegate to each site responsibility for maintaining names within its partition. The topmost level of the hierarchy divides the namespace and delegates authority for each division; it need not be bothered by changes within a division.

The syntax of hierarchically assigned names often reflects the hierarchical delegation of authority used to assign them. As an example, consider a namespace with names of the form:

local.site

where *site* is the site name authorized by the central authority, *local* is the part of a name controlled by the site, and the period† (“.”) is a delimiter used to separate them. When the topmost authority approves adding a new site, *X*, it adds *X* to the list of valid sites and delegates to site *X* authority for all names that end in “.*X*”.

24.6 Subset Authority

In a hierarchical namespace, authority may be further subdivided at each level. In our example of partition by sites, the site itself may consist of several administrative groups, and the site authority may choose to subdivide its namespace among the groups. The idea is to keep subdividing the namespace until each subdivision is small enough to be manageable.

Syntactically, subdividing the namespace introduces another partition of the name. For example, adding a *group* subdivision to names already partitioned by site produces the following name syntax:

local.group.site

Because the topmost level delegates authority, group names do not have to agree among all sites. A university site might choose group names like *engineering*, *science*, and *arts*, while a corporate site might choose group names like *production*, *accounting*, and *personnel*.

The U.S. telephone system provides another example of a hierarchical naming syntax. The 10 digits of a phone number have been partitioned into a 3-digit *area code*, 3-digit *exchange*, and 4-digit *subscriber number* within the exchange. Each exchange has authority for assigning subscriber numbers within its piece of the namespace. Although it is possible to group arbitrary subscribers into exchanges and to group arbitrary exchanges into area codes, the assignment of telephone numbers is not capricious; they are carefully chosen to make it easy to route phone calls across the telephone network.

†In domain names, the period delimiter is pronounced “dot.”

The telephone example is important because it illustrates a key distinction between the hierarchical naming scheme used in a TCP/IP internet and other hierarchies: partitioning the set of machines owned by an organization along lines of authority does not necessarily imply partitioning by physical location. For example, it could be that at some university, a single building houses the mathematics department as well as the computer science department. It might even turn out that although the machines from these two groups fall under completely separate administrative domains, they connect to the same physical network. It also may happen that a single group owns machines on several physical networks. For these reasons, the TCP/IP naming scheme allows arbitrary delegation of authority for the hierarchical namespace without regard to physical connections. The concept can be summarized:

In a TCP/IP internet, hierarchical machine names are assigned according to the structure of organizations that obtain authority for parts of the namespace, not necessarily according to the structure of the physical network interconnections.

Of course, at many sites the organizational hierarchy corresponds with the structure of physical network interconnections. At a large university, for example, most departments have their own local area network. If the department is assigned part of the naming hierarchy, all machines that have names in its part of the hierarchy will also connect to a single physical network.

24.7 Internet Domain Names

The mechanism that implements a machine name hierarchy for TCP/IP internets is called the *Domain Name System (DNS)*. DNS has two, conceptually independent aspects. The first is abstract: it specifies the name syntax and rules for delegating authority over names. The second is concrete: it specifies the implementation of a distributed computing system that efficiently maps names to addresses. This section considers the name syntax, and later sections examine the implementation.

The domain name system uses a hierarchical naming scheme known as *domain names*. As in our earlier examples, a domain name consists of a sequence of subnames separated by a delimiter character, the period. In our examples we said that individual sections of the name might represent sites or groups, but the domain system simply calls each section a *label*. Thus, the domain name

cs.purdue.edu

contains three labels: *cs*, *purdue*, and *edu*. Any suffix of a label in a domain name is also called a *domain*. In the above example the lowest level domain is *cs.purdue.edu*, (the domain name for the Computer Science Department at Purdue University), the second level domain is *purdue.edu* (the domain name for Purdue University), and the

top-level domain is *edu* (the domain name for educational institutions). As the example shows, domain names are written with the local label first and the top domain last. As we will see, writing them in this order makes it possible to compress messages that contain multiple domain names.

24.8 Official And Unofficial Internet Domain Names

In theory, the domain name standard specifies an abstract hierarchical namespace with arbitrary values for labels. Because the domain system dictates only the form of names and not their actual values, it is possible for any group that builds an instance of the domain system to choose labels for all parts of its hierarchy. For example, a private company can establish a domain hierarchy in which the top-level labels specify corporate subsidiaries, the next level labels specify corporate divisions, and the lowest level labels specify departments.

However, most users of the domain technology follow the hierarchical labels used by the official Internet domain system. There are two reasons. First, as we will see, the Internet scheme is both comprehensive and flexible. It can accommodate a wide variety of organizations, and allows each group to choose between geographical or organizational naming hierarchies. Second, most sites follow the Internet scheme so they can attach their TCP/IP installations to the global Internet without changing names. Because the Internet naming scheme dominates almost all uses of the domain name system, examples throughout the remainder of this chapter have labels taken from the Internet naming hierarchy. Readers should remember that, although they are most likely to encounter these particular labels, the domain name system technology can be used with other labels if desired.

The Internet authority has chosen to partition its top level into the domains listed in Figure 24.1†.

Domain Name	Meaning
COM	Commercial organizations
EDU	Educational institutions (4-year)
GOV	Government institutions
MIL	Military groups
NET	Major network support centers
ORG	Organizations other than those above
ARPA	Temporary ARPANET domain (obsolete)
INT	International organizations
country code	Each country (geographic scheme)

Figure 24.1 The top-level Internet domains and their meanings. Although labels are shown in upper case, domain name system comparisons are insensitive to case, so *EDU* is equivalent to *edu*.

†The following additional top-level domains have been proposed, but not formally adopted: *FIRM*, *STORE*, *WEB*, *ARTS*, *REC*, *INFO*, and *NOM*.

Conceptually, the top-level names permit two completely different naming hierarchies: geographic and organizational. The geographic scheme divides the universe of machines by country. Machines in the United States fall under the top-level domain *US*; when a foreign country wants to register machines in the domain name system, the central authority assigns the country a new top-level domain with the country's international standard 2-letter identifier as its label. The authority for the US domain has chosen to divide it into one second-level domain per state. For example, the domain for the state of Virginia is

va.us

As an alternative to the geographic hierarchy, the top-level domains also allow organizations to be grouped by organizational type. When an organization wants to participate in the domain naming system, it chooses how it wishes to be registered and requests approval. The central authority reviews the application and assigns the organization a subdomain† under one of the existing top-level domains. For example, it is possible for a university to register itself as a second-level domain under *EDU* (the usual practice), or to register itself under the state and country in which it is located. So far, few organizations have chosen the geographic hierarchy; most prefer to register under *COM*, *EDU*, *MIL*, or *GOV*. There are two reasons. First, geographic names are longer and therefore more difficult to type. Second, geographic names are much more difficult to discover or guess. For example, Purdue University is located in West Lafayette, Indiana. While a user could easily guess an organizational name, like *purdue.edu*, a geographic name is often difficult to guess because it is usually an abbreviation, like *laf.in.us*.

Another example may help clarify the relationship between the naming hierarchy and authority for names. A machine named *xinu* in the Computer Science Department at Purdue University has the official domain name

xinu.cs.purdue.edu

The machine name was approved and registered by the local network manager in the Computer Science Department. The department manager had previously obtained authority for the subdomain *cs.purdue.edu* from a university network authority, who had obtained permission to manage the subdomain *purdue.edu* from the Internet authority. The Internet authority retains control of the *edu* domain, so new universities can only be added with its permission. Similarly, the university network manager at Purdue University retains authority for the *purdue.edu* subdomain, so new third-level domains may only be added with the manager's permission.

Figure 24.2 illustrates a small part of the Internet domain name hierarchy. As the figure shows, Digital Equipment Corporation, a commercial organization, registered as *dec.com*, Purdue University registered as *purdue.edu*, and the National Science Foundation, a government agency, registered as *nsf.gov*. In contrast, the Corporation for National Research Initiatives chose to register under the geographic hierarchy as *cnri.reston.va.us‡*.

†The standard does not define the term “subdomain.” We have chosen to use it because its analogy to “subset” helps clarify the relationship among domains.

‡Interestingly, CNRI also registered using the name *nri.reston.va.us*.

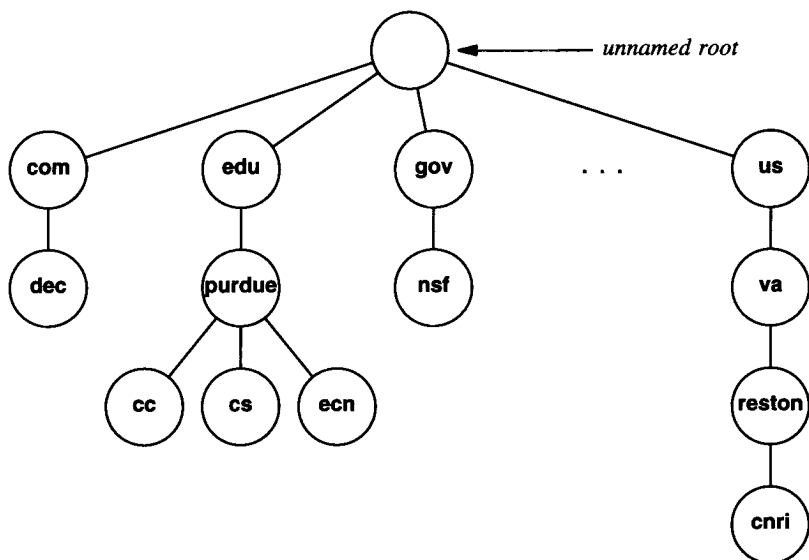


Figure 24.2 A small part of the Internet domain name hierarchy (tree). In practice, the tree is broad and flat; most host entries appear by the fifth level.

24.9 Named Items And Syntax Of Names

The domain name system is quite general because it allows multiple naming hierarchies to be embedded in one system. To allow clients to distinguish among multiple types of entries, each named item stored in the system is assigned a *type* that specifies whether it is the address of a machine, a mailbox, a user, and so on. When a client asks the domain system to resolve a name, it must specify the type of answer desired. For example, when an electronic mail application uses the domain system to resolve a name, it specifies that the answer should be the address of a *mail exchanger*. A remote login application specifies that it seeks a machine's IP address. It is important to understand the following:

*A given name may map to more than one item in the domain system.
The client specifies the type of object desired when resolving a name,
and the server returns objects of that type.*

In addition to specifying the type of answer sought, the domain system allows the client to specify the protocol family to use. The domain system partitions the entire set of names by *class*, allowing a single database to store mappings for multiple protocol suites†.

†In practice, few domain servers use multiple protocol suites.

The syntax of a name does not determine what type of object it names or the class of protocol suite. In particular, the number of labels in a name does not determine whether the name refers to an individual object (machine) or a domain. Thus, in our example, it is possible to have a machine named

gwen.purdue.edu

even though

cs.purdue.edu

names a subdomain. We can summarize this important point:

One cannot distinguish the names of subdomains from the names of individual objects or the type of an object using only the domain name syntax.

24.10 Mapping Domain Names To Addresses

In addition to the rules for name syntax and delegation of authority, the domain name scheme includes an efficient, reliable, general purpose, distributed system for mapping names to addresses. The system is distributed in the technical sense, meaning that a set of servers operating at multiple sites cooperatively solve the mapping problem. It is efficient in the sense that most names can be mapped locally; only a few require internet traffic. It is general purpose because it is not restricted to machine names (although we will use that example for now). Finally, it is reliable in that no single machine failure will prevent the system from operating correctly.

The domain mechanism for mapping names to addresses consists of independent, cooperative systems called *name servers*. A name server is a server program that supplies name-to-address translation, mapping from domain names to IP addresses. Often, server software executes on a dedicated processor, and the machine itself is called the name server. The client software, called a *name resolver*, uses one or more name servers when translating a name.

The easiest way to understand how domain servers work is to imagine them arranged in a tree structure that corresponds to the naming hierarchy, as Figure 24.3 illustrates. The root of the tree is a server that recognizes the top-level domains and knows which server resolves each domain. Given a name to resolve, the root can choose the correct server for that name. At the next level, a set of name servers each provide answers for one top-level domain (e.g., *edu*). A server at this level knows which servers can resolve each of the subdomains under its domain. At the third level of the tree, name servers provide answers for subdomains (e.g., *purdue* under *edu*). The conceptual tree continues with one server at each level for which a subdomain has been defined.

Links in the conceptual tree do not indicate physical network connections. Instead, they show which other name servers a given server knows and contacts. The servers themselves may be located at arbitrary locations on an internet. Thus, the tree of servers is an abstraction that uses an internet for communication.

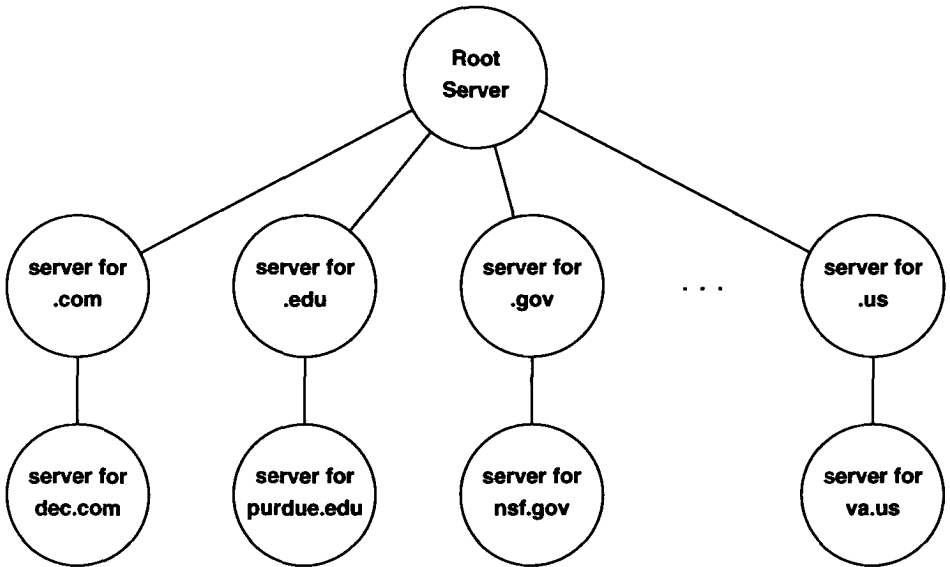


Figure 24.3 The conceptual arrangement of domain name servers in a tree that corresponds to the naming hierarchy. In theory, each server knows the addresses of all lower-level servers for all subdomains within the domain it handles.

If servers in the domain system worked exactly as our simplistic model suggests, the relationship between connectivity and authorization would be quite simple. When authority was granted for a subdomain, the organization requesting it would need to establish a domain name server for that subdomain and link it into the tree.

In practice, the relationship between the naming hierarchy and the tree of servers is not as simple as our model implies. The tree of servers has few levels because a single physical server can contain all of the information for large parts of the naming hierarchy. In particular, organizations often collect information from all of their subdomains into a single server. Figure 24.4 shows a more realistic organization of servers for the naming hierarchy of Figure 24.2.

A root server contains information about the root and top-level domains, and each organization uses a single server for its names. Because the tree of servers is shallow, at most two servers need to be contacted to resolve a name like *xinu.cs.purdue.edu*: the root server and the server for domain *purdue.edu* (i.e., the root server knows which

server handles *purdue.edu*, and the entire domain information for Purdue resides in one server).

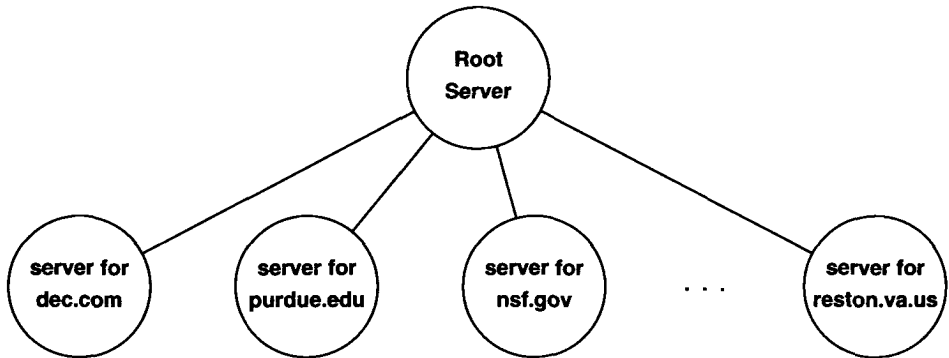


Figure 24.4 A realistic organization of servers for the naming hierarchy of Figure 24.2. Because the tree is broad and flat, few servers need to be contacted when resolving a name.

24.11 Domain Name Resolution

Although the conceptual tree makes understanding the relationship between servers easy, it hides several subtle details. Looking at the name resolution algorithm will help explain them. Conceptually, domain name resolution proceeds top-down, starting with the root name server and proceeding to servers located at the leaves of the tree. There are two ways to use the domain name system: by contacting name servers one at a time or asking the name server system to perform the complete translation. In either case, the client software forms a domain name query that contains the name to be resolved, a declaration of the class of the name, the type of answer desired, and a code that specifies whether the name server should translate the name completely. It sends the query to a name server for resolution.

When a domain name server receives a query, it checks to see if the name lies in the subdomain for which it is an authority. If so, it translates the name to an address according to its database, and appends an answer to the query before sending it back to the client. If the name server cannot resolve the name completely, it checks to see what type of interaction the client specified. If the client requested complete translation (*recursive resolution*, in domain name terminology), the server contacts a domain name server that can resolve the name and returns the answer to the client. If the client requested non-recursive resolution (*iterative resolution*), the name server cannot supply an answer. It generates a reply that specifies the name server the client should contact next to resolve the name.

How does a client find a name server at which to begin the search? How does a name server find other name servers that can answer questions when it cannot? The answers are simple. A client must know how to contact at least one name server. To ensure that a domain name server can reach others, the domain system requires that each server know the address of at least one root server†. In addition, a server may know the address of a server for the domain immediately above it (called the *parent*).

Domain name servers use a well-known protocol port for all communication, so clients know how to communicate with a server once they know the IP address of the machine in which the server executes. There is no standard way for hosts to locate a machine in the local environment on which a name server runs; that is left to whoever designs the client software‡.

In some systems, the address of the machine that supplies domain name service is bound into application programs at compile time, while in others, the address is configured into the operating system at startup. In others, the administrator places the address of a server in a file on secondary storage.

24.12 Efficient Translation

Although it may seem natural to resolve queries by working down the tree of name servers, it can lead to inefficiencies for three reasons. First, most name resolution refers to local names, those found within the same subdivision of the namespace as the machine from which the request originates. Tracing a path through the hierarchy to contact the local authority would be inefficient. Second, if each name resolution always started by contacting the topmost level of the hierarchy, the machine at that point would become overloaded. Third, failure of machines at the topmost levels of the hierarchy would prevent name resolution, even if the local authority could resolve the name. The telephone number hierarchy mentioned earlier helps explain. Although telephone numbers are assigned hierarchically, they are resolved in a bottom-up fashion. Because the majority of telephone calls are local, they can be resolved by the local exchange without searching the hierarchy. Furthermore, calls within a given area code can be resolved without contacting sites outside the area code. When applied to domain names, these ideas lead to a two-step name resolution mechanism that preserves the administrative hierarchy but permits efficient translation.

We have said that most queries to name servers refer to local names. In the two-step name resolution process, resolution begins with the local name server. If the local server cannot resolve a name, the query must then be sent to another server in the domain system.

†For reliability, there are multiple servers for each node in the domain server tree; the root server is further replicated to provide load balancing.

‡See BOOTP/DHCP in Chapter 23 for one possible approach.

24.13 Caching: The Key To Efficiency

The cost of lookup for nonlocal names can be extremely high if resolvers send each query to the root server. Even if queries could go directly to the server that has authority for the name, name lookup can present a heavy load to an internet. Thus, to improve the overall performance of a name server system, it is necessary to lower the cost of lookup for nonlocal names.

Internet name servers use *name caching* to optimize search costs. Each server maintains a cache of recently used names as well as a record of where the mapping information for that name was obtained. When a client asks the server to resolve a name, the server first checks to see if it has authority for the name according to the standard procedure. If not, the server checks its cache to see if the name has been resolved recently. Servers report cached information to clients, but mark it as a *nonauthoritative* binding, and give the domain name of the server, *S*, from which they obtained the binding. The local server also sends along additional information that tells the client the binding between *S* and an IP address. Therefore, clients receive answers quickly, but the information may be out-of-date. If efficiency is important, the client will choose to accept the nonauthoritative answer and proceed. If accuracy is important, the client will choose to contact the authority and verify that the binding between name and address is still valid.

Caching works well in the domain name system because name to address bindings change infrequently. However, they do change. If servers cached information the first time it was requested and never changed it, entries in the cache could become incorrect. To keep the cache correct, servers time each entry and dispose of entries that exceed a reasonable time. When the server is asked for the information after it has removed the entry from the cache, it must go back to the authoritative source and obtain the binding again. More important, servers do not apply a single fixed timeout to all entries, but allow the authority for an entry to configure its timeout. Whenever an authority responds to a request, it includes a *Time To Live* (TTL) value in the response that specifies how long it guarantees the binding to remain. Thus, authorities can reduce network overhead by specifying long timeouts for entries that they expect to remain unchanged, while improving correctness by specifying short timeouts for entries that they expect to change frequently.

Caching is important in hosts as well as in local domain name servers. Many timesharing systems run a complex form of resolver code that attempts to provide even more efficiency than the server system. The host downloads the complete database of names and addresses from a local domain name server at startup, maintains its own cache of recently used names, and uses the server only when names are not found. Naturally, a host that maintains a copy of the local server database must check with the server periodically to obtain new mappings, and the host must remove entries from its cache after they become invalid. However, most sites have little trouble maintaining consistency because domain names change so infrequently.

Keeping a copy of the local server's database in each host has several advantages. Obviously, it makes name resolution on local hosts extremely fast because it means the

host can resolve names without any network activity. It also means that the local site has protection in case the local name server fails. Finally, it reduces the computational load on the name server, and makes it possible for a given server to supply names to more machines.

24.14 Domain Server Message Format

Looking at the details of messages exchanged between clients and domain name servers will help clarify how the system operates from the view of a typical application program. We assume that a user invokes an application program and supplies the name of a machine with which the application must communicate. Before it can use protocols like TCP or UDP to communicate with the specified machine, the application program must find the machine's IP address. It passes the domain name to a local resolver and requests an IP address. The local resolver checks its cache and returns the answer if one is present. If the local resolver does not have an answer, it formats a message and sends it to the server (i.e., it becomes a client). Although our example only involves one name, the message format allows a client to ask multiple questions in a single message. Each question consists of a domain name for which the client seeks an IP address, a specification of the query class (i.e., *internet*), and the type of object desired (e.g., *address*). The server responds by returning a similar message that contains answers to the questions for which the server has bindings. If the server cannot answer all questions, the response will contain information about other name servers that the client can contact to obtain the answers.

Responses also contain information about the servers that are authorities for the replies and the IP addresses of those servers. Figure 24.5 shows the message format. As the figure shows, each message begins with a fixed header. The header contains a unique *IDENTIFICATION* field that the client uses to match responses to queries, and a *PARAMETER* field that specifies the operation requested and a response code. Figure 24.6 gives the interpretation of bits in the *PARAMETER* field.

The fields labeled *NUMBER OF* each give a count of entries in the corresponding sections that occur later in the message. For example, the field labeled *NUMBER OF QUESTIONS* gives the count of entries that appear in the *QUESTION SECTION* of the message.

The *QUESTION SECTION* contains queries for which answers are desired. The client fills in only the question section; the server returns the questions and answers in its response. Each question consists of a *QUERY DOMAIN NAME* followed by *QUERY TYPE* and *QUERY CLASS* fields, as Figure 24.7 shows.

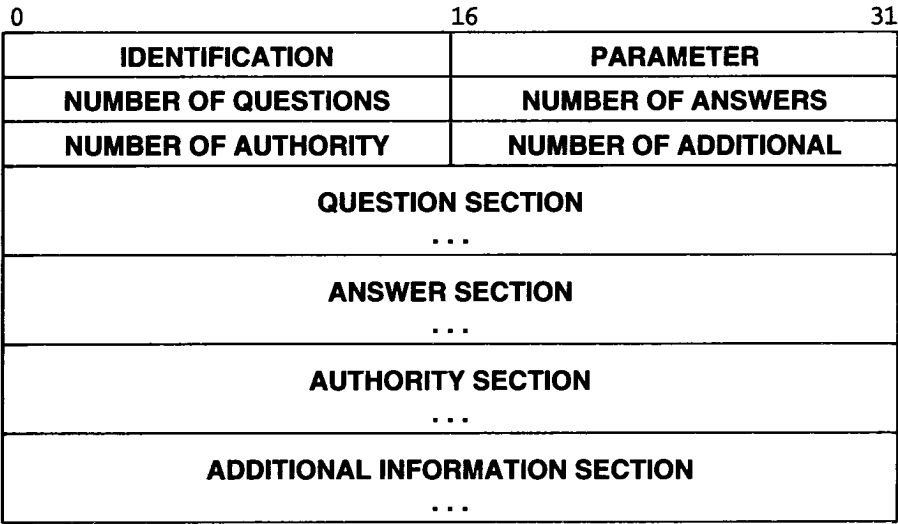


Figure 24.5 Domain name server message format. The question, answer, authority, and additional information sections are variable length.

Bit of PARAMETER field	Meaning
0	Operation: 0 Query 1 Response
1-4	Query Type: 0 Standard 1 Inverse 2 Completion 1 (now obsolete) 3 Completion 2 (now obsolete)
5	Set if answer authoritative
6	Set if message truncated
7	Set if recursion desired
8	Set if recursion available
9-11	Reserved
12-15	Response Type: 0 No error 1 Format error in query 2 Server failure 3 Name does not exist

Figure 24.6 The meaning of bits of the *PARAMETER* field in a domain name server message. Bits are numbered left to right starting at 0.

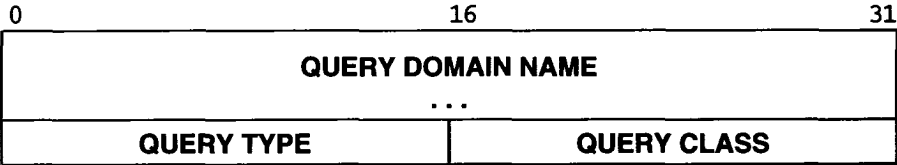


Figure 24.7 The format of entries in the *QUESTION SECTION* of a domain name server message. The domain name is variable length. Clients fill in the questions; servers return them along with answers.

Although the *QUERY DOMAIN NAME* field has variable length, we will see in the next section that the internal representation of domain names makes it possible for the receiver to know the exact length. The *QUERY TYPE* encodes the type of the question (e.g., whether the question refers to a machine name or a mail address). The *QUERY CLASS* field allows domain names to be used for arbitrary objects because official Internet names are only one possible class. It should be noted that, although the diagram in Figure 24.5 follows our convention of showing formats in 32-bit multiples, the *QUERY DOMAIN NAME* field may contain an arbitrary number of octets. No padding is used. Therefore, messages to or from domain name servers may contain an odd number of octets.

In a domain name server message, each of the *ANSWER SECTION*, *AUTHORITY SECTION*, and *ADDITIONAL INFORMATION SECTION* consists of a set of *resource records* that describe domain names and mappings. Each resource record describes one name. Figure 24.8 shows the format.

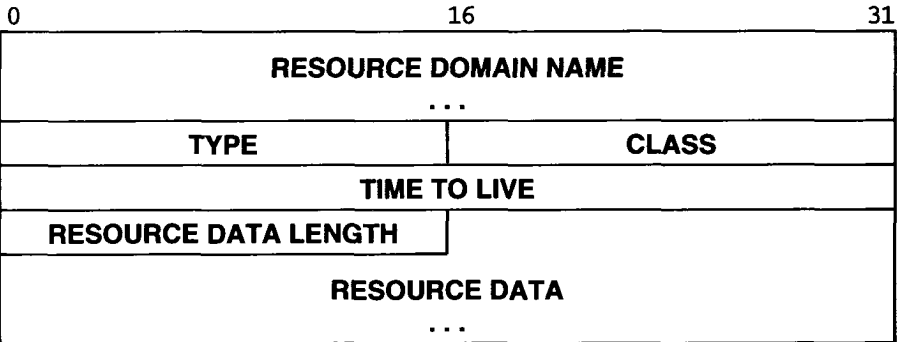


Figure 24.8 The format of resource records used in later sections of messages returned by domain name servers.

The *RESOURCE DOMAIN NAME* field contains the domain name to which this resource record refers. It may be an arbitrary length. The *TYPE* field specifies the type of the data included in the resource record; the *CLASS* field specifies the data's class. The *TIME TO LIVE* field contains a 32-bit integer that specifies the number of seconds information in this resource record can be cached. It is used by clients who have requested a name binding and may want to cache the results. The last two fields contain the results of the binding, with the *RESOURCE DATA LENGTH* field specifying the count of octets in the *RESOURCE DATA* field.

24.15 Compressed Name Format

When represented in a message, domain names are stored as a sequence of labels. Each label begins with an octet that specifies its length. Thus, the receiver reconstructs a domain name by repeatedly reading a 1-octet length, n , and then reading a label n octets long. A length octet containing zero marks the end of the name.

Domain name servers often return multiple answers to a query and, in many cases, suffixes of the domain overlap. To conserve space in the reply packet, the name servers compress names by storing only one copy of each domain name. When extracting a domain name from a message, the client software must check each segment of the name to see whether it consists of a literal string (in the format of a 1-octet count followed by the characters that make up the name) or a pointer to a literal string. When it encounters a pointer, the client must follow the pointer to a new place in the message to find the remainder of the name.

Pointers always occur at the beginning of segments and are encoded in the count byte. If the top two bits of the 8-bit segment count field are 1s, the client must take the next 14 bits as an integer pointer. If the top two bits are zero, the next 6 bits specify the number of characters in the label that follow the count octet.

24.16 Abbreviation Of Domain Names

The telephone number hierarchy illustrates another useful feature of local resolution, *name abbreviation*. Abbreviation provides a method of shortening names when the resolving process can supply part of the name automatically. Normally, a subscriber omits the area code when dialing a local telephone number. The resulting digits form an abbreviated name assumed to lie within the same area code as the subscriber's phone. Abbreviation also works well for machine names. Given a name like *xyz*, the resolving process can assume it lies in the same local authority as the machine on which it is being resolved. Thus, the resolver can supply missing parts of the name automati-

cally. For example, within the Computer Science Department at Purdue, the abbreviated name

xinu

is equivalent to the full domain name

xinu.cs.purdue.edu

Most client software implements abbreviations with a *domain suffix list*. The local network manager configures a list of possible suffixes to be appended to names during lookup. When a resolver encounters a name, it steps through the list, appending each suffix and trying to look up the resulting name. For example, the suffix list for the Computer Science Department at Purdue includes:

.cs.purdue.edu
.cc.purdue.edu
.purdue.edu
null

Thus, local resolvers first append *cs.purdue.edu* onto the name *xinu*. If that lookup fails, they append *cc.purdue.edu* onto the name and look that up. The last suffix in the example list is the null string, meaning that if all other lookups fail, the resolver will attempt to look up the name with no suffix. Managers can use the suffix list to make abbreviation convenient or to restrict application programs to local names.

We said that the client takes responsibility for the expansion of such abbreviations, but it should be emphasized that such abbreviations are not part of the domain name system itself. The domain system only allows lookup of a fully specified domain name. As a consequence, programs that depend on abbreviations may not work correctly outside the environment in which they were built. We can summarize:

The domain name system only maps full domain names into addresses; abbreviations are not part of the domain name system itself, but are introduced by client software to make local names convenient for users.

24.17 Inverse Mappings

We said that the domain name system can provide mappings other than machine name to IP address. *Inverse queries* allow the client to ask a server to map “backwards” by taking an answer and generating the question that would produce that answer. Of course, not all answers have a unique question. Even when they do, a server may not be able to provide it. Although inverse queries have been part of the domain system since it was first specified, they are generally not used because there is often no way to find the server that can resolve the query without searching the entire set of servers.

24.18 Pointer Queries

One form of inverse mapping is so obviously needed that the domain system supports a special domain and a special form of question called a *pointer query* to answer it. In a pointer query, the question presented to a domain name server specifies an IP address encoded as a printable string in the form of a domain name (i.e., a textual representation of digits separated by periods). A pointer query requests the name server to return the correct domain name for the machine with the specified IP address. Pointer queries are especially useful for diskless machines because they allow the system to obtain a high-level name given only an IP address. (We have already seen in Chapter 6 how a diskless machine can obtain its IP address.)

Pointer queries are not difficult to generate. If we think of an IP address written in dotted-decimal form, it has the following format:

aaa.bbb.ccc.ddd

To form a pointer query, the client rearranges the dotted decimal representation of the address into a string of the form:

ddd.ccc.bbb.aaa.in-addr.arpa

The new form is a name in the special *in-addr.arpa* domain†. Because the local name server may not be the authority for either the *arpa* domain or the *in-addr.arpa* domain, it may need to contact other name servers to complete the resolution. To make the resolution of pointer queries efficient, the Internet root domain servers maintain a database of valid IP addresses along with information about domain name servers that can resolve each address.

24.19 Object Types And Resource Record Contents

We have mentioned that the domain name system can be used for translating a domain name to a mail exchanger address as well as for translating a host name to an IP address. The domain system is quite general in that it can be used for arbitrary hierarchical names. For example, one might decide to store the names of available computational services along with a mapping from each name to the telephone number to call to find out about the corresponding service. Or one might store names of protocol products along with a mapping to the names and addresses of vendors that offer such products.

Recall that the system accommodates a variety of mappings by including a *type* in each resource record. When sending a request, a client must specify the type in its query‡; servers specify the data type in all resource records they return. The type determines the contents of the resource record according to the table in Figure 24.9

†The octets of the IP address must be reversed when forming a domain name because IP addresses have the most significant octets first while domain names have the least-significant octets first.

‡Queries can specify a few additional types (e.g., there is a query type that requests all resource records).

Type	Meaning	Contents
A	Host Address	32-bit IP address
CNAME	Canonical Name	Canonical domain name for an alias
HINFO	CPU & OS	Name of CPU and operating system
MINFO	Mailbox info	Information about a mailbox or mail list
MX	Mail Exchanger	16-bit preference and name of host that acts as mail exchanger for the domain
NS	Name Server	Name of authoritative server for domain
PTR	Pointer	Domain name (like a symbolic link)
SOA	Start of Authority	Multiple fields that specify which parts of the naming hierarchy a server implements
TXT	Arbitrary text	Uninterpreted string of ASCII text

Figure 24.9 Domain name system resource record types.

Most data is of type *A*, meaning that it consists of the name of a host attached to the Internet along with the host's IP address. The second most useful domain type, *MX*, is assigned to names used for electronic mail exchangers. It allows a site to specify multiple hosts that are each capable of accepting mail. When sending electronic mail, the user specifies an electronic mail address in the form *user@domain-part*. The mail system uses the domain name system to resolve *domain-part* with query type *MX*. The domain system returns a set of resource records that each contain a preference field and a host's domain name. The mail system steps through the set from highest preference to lowest (lower numbers mean higher preference). For each *MX* resource record, the mailer extracts the domain name and uses a type *A* query to resolve that name to an IP address. It then tries to contact the host and deliver mail. If the host is unavailable, the mailer will continue trying other hosts on the list.

To make lookup efficient, a server always returns additional bindings that it knows in the *ADDITIONAL INFORMATION SECTION* of a response. In the case of *MX* records, a domain server can use the *ADDITIONAL INFORMATION SECTION* to return type *A* resource records for domain names reported in the *ANSWER SECTION*. Doing so substantially reduces the number of queries a mailer sends to its domain server.

24.20 Obtaining Authority For A Subdomain

Before an institution is granted authority for an official second-level domain, it must agree to operate a domain name server that meets Internet standards. Of course, a domain name server must obey the protocol standards that specify message formats and the rules for responding to requests. The server must also know the addresses of servers that handle each subdomain (if any exist) as well as the address of at least one root server.

In practice, the domain system is much more complex than we have outlined. In most cases, a single physical server can handle more than one part of the naming hierarchy. For example, a single name server at Purdue University handles both the second-level domain *purdue.edu* as well as the geographic domain *laf.in.us*. A subtree of names managed by a given name server forms a *zone of authority*. Another practical complication arises because servers must be able to handle many requests, even though some requests take a long time to resolve. Usually, servers support concurrent activity, allowing work to proceed on later requests while earlier ones are being processed. Handling requests concurrently is especially important when the server receives a recursive request that forces it to send the request on to another server for resolution.

Server implementation is also complicated because the Internet authority requires that the information in every domain name server be replicated. Information must appear in at least two servers that do not operate on the same computer. In practice, the requirements are quite stringent: the servers must have no single common point of failure. Avoiding common points of failure means that the two name servers cannot both attach to the same network; they cannot even obtain electrical power from the same source. Thus, to meet the requirements, a site must find at least one other site that agrees to operate a backup name server. Of course, at any point in the tree of servers, a server must know how to locate both the primary and backup name servers for subdomains, and it must direct queries to a backup name server if the primary server is unavailable.

24.21 Summary

Hierarchical naming systems allow delegation of authority for names, making it possible to accommodate an arbitrarily large set of names without overwhelming a central site with administrative duties. Although name resolution is separate from delegation of authority, it is possible to create hierarchical naming systems in which resolution is an efficient process that starts at the local server even though delegation of authority always flows from the top of the hierarchy downward.

We examined the Internet domain name system (DNS) and saw that it offers a hierarchical naming scheme. DNS uses distributed lookup in which domain name servers map each domain name to an IP address or mail exchanger address. Clients begin by trying to resolve names locally. When the local server cannot resolve the name, the client must choose to work through the tree of name servers iteratively or request the local name server to do it recursively. Finally, we saw that the domain name system supports a variety of bindings including bindings from IP addresses to high-level names.

FOR FURTHER STUDY

Mockapetris [RFC 1034] discusses Internet domain naming in general, giving the overall philosophy, while Mockapetris [RFC 1035] provides a protocol standard for the domain name system. Mockapetris [RFC 1101] discusses using the domain name system to encode network names and proposes extensions useful for other mappings. Postel and Reynolds [RFC 920] states the requirements that an Internet domain name server must meet. Stahl [RFC 1032] gives administrative guidelines for establishing a domain, and Lottor [RFC 1033] provides guidelines for operating a domain name server. Eastlake [RFC 2535] presents security extensions. Partridge [RFC 974] relates domain naming to electronic mail addressing. Finally, Lottor [RFC 1296] provides an interesting summary of Internet growth obtained by walking the domain name tree.

EXERCISES

- 24.1 Machine names should not be bound into the operating system at compile time. Explain why.
- 24.2 Would you prefer to use a machine that obtained its name from a remote file or from a name server? Why?
- 24.3 Why should each name server know the IP address of its parent instead of the domain name of its parent?
- 24.4 Devise a naming scheme that tolerates changes to the naming hierarchy. As an example, consider two large companies that each have an independent naming hierarchy, and suppose the companies merge. Can you arrange to have all previous names still work correctly?
- 24.5 Read the standard and find out how the domain name system uses *SOA* records.
- 24.6 The Internet domain name system can also accommodate mailbox names. Find out how.
- 24.7 The standard suggests that when a program needs to find the domain name associated with an IP address, it should send an inverse query to the local server first and use domain *in-addr.arpa* only if that fails. Why?
- 24.8 How would you accommodate abbreviations in a domain naming scheme? As an example, show two sites that are both registered under *.edu* and a top level server. Explain how each site would treat each type of abbreviation.
- 24.9 Obtain the official description of the domain name system and build a client program. Look up the name *merlin.cs.purdue.edu*.
- 24.10 Extend the exercise above to include a pointer query. Try looking up the domain name for address *128.10.2.3*.
- 24.11 Find a copy of the program *nslookup*, and use it to look up the names in the two previous exercises.

- 24.12** If we extended the domain name syntax to include a dot after the top-level domain, names and abbreviations would be unambiguous. What are the advantages and disadvantages of the extension?
- 24.13** Read the RFCs on the domain name system. What are the maximum and minimum possible values a DNS server can store in the *TIME-TO-LIVE* field of a resource record?
- 24.14** Should the domain name system permit partial match queries (i.e. a wildcard as part of a name)? Why or why not?
- 24.15** The Computer Science Department at Purdue University chose to place the following type *A* resource record entry in its domain name server:

`localhost.cs.purdue.edu     127.0.0.1`

Explain what will happen if a remote site tries to *ping* a machine with domain name *localhost.cs.purdue.edu*.

Applications: Remote Login (TELNET, Rlogin)

25.1 Introduction

This chapter and the next five continue our exploration of internetworking by examining high-level internet services and the protocols that support them. These services form an integral part of TCP/IP. They determine how users perceive an internet and demonstrate the power of the technology.

We will learn that high-level services provide increased communication functionality, and allow users and programs to interact with automated services on remote machines and with remote users. We will see that high-level protocols are implemented with application programs, and will learn how they depend on the network level services described in previous chapters. This chapter begins by examining remote login.

25.2 Remote Interactive Computing

We have already seen how the client-server model can provide specific computational services like a time-of-day service to multiple machines. Reliable stream protocols like TCP make possible interactive use of remote machines as well. For example, imagine building a server that provides a remote text editing service. To implement an editing service, we need a server that accepts requests to edit a file and a client to make such requests. To invoke the remote editor service, a user executes the client program. The client establishes a TCP connection to the server, and then begins sending keystrokes to the server and reading output that the server sends back.

How can our imagined remote interactive editing service be generalized? The problem with using one server for each computational service is that machines quickly become swamped with server processes. We can eliminate most specialized servers and provide more generality by allowing the user to establish a login session on the remote machine and then execute commands. With a *remote login* facility, users have access to all the commands available on the remote system, and system designers need not provide specialized servers.

Of course, providing remote login may not be simple. Computer systems that are designed without considering networking expect login sessions only from a directly connected keyboard and display. On such a computer, adding a remote login server requires modifying the machine's operating system. Building interactive client software may also be difficult. Consider, for example, a system that assigns special meaning to some keystrokes. If the local system interprets Control-C to mean "abort the currently executing command process," it may be impossible to pass Control-C to the remote machine. If the client does pass Control-C to the remote site, it may be impossible to abort the local client process.

Despite the technical difficulties, system programmers have managed to build remote login server software for most operating systems and to construct application programs that act as clients. Often, the client software overrides the local interpretation of all keys except one, allowing a user to interact with the remote machine exactly as one would from a locally connected terminal. The single key exception provides a way for a user to escape to the local environment and control the client (e.g., to abort the client). In addition, some remote login protocols recognize a set of *trusted hosts*, permitting remote login from such hosts without verifying passwords, and others achieve security by encrypting all transmissions.

25.3 TELNET Protocol

The TCP/IP protocol suite includes a simple remote terminal protocol called *TELNET* that allows a user to log into a computer across an internet. TELNET establishes a TCP connection, and then passes keystrokes from the user's keyboard directly to the remote computer as if they had been typed on a keyboard attached to the remote machine. TELNET also carries output from the remote machine back to the user's screen. The service is called *transparent* because it gives the appearance that the user's keyboard and display attach directly to the remote machine.

Although TELNET is not as sophisticated as some remote terminal protocols, it is widely available. Usually, TELNET client software allows the user to specify a remote machine either by giving its domain name or IP address. Because it accepts IP addresses, TELNET can be used with hosts even if a name-to-address binding cannot be established (e.g., when domain naming software is being debugged).

TELNET offers three basic services. First, it defines a *network virtual terminal* that provides a standard interface to remote systems. Client programs do not have to understand the details of all possible remote systems; they are built to use the standard

interface. Second, TELNET includes a mechanism that allows the client and server to negotiate options, and it provides a set of standard options (e.g., one of the options controls whether data passed across the connection uses the standard 7-bit ASCII character set or an 8-bit character set). Finally, TELNET treats both ends of the connection symmetrically. In particular, TELNET does not force client input to come from a keyboard, nor does it force the client to display output on a screen. Thus, TELNET allows an arbitrary program to become a client. Furthermore, either end can negotiate options.

Figure 25.1 illustrates how application programs implement a TELNET client and server.

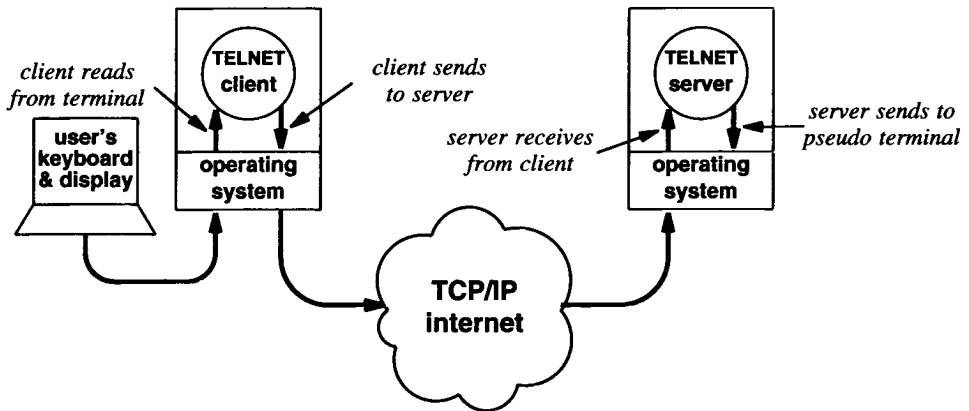


Figure 25.1 The path of data in a TELNET remote terminal session as it travels from the user's keyboard to the remote operating system. Adding a TELNET server to a timesharing system usually requires modifying the operating system.

As the figure shows, when a user invokes TELNET, an application program on the user's machine becomes the client. The client establishes a TCP connection to the server over which they will communicate. Once the connection has been established, the client accepts keystrokes from the user's keyboard and sends them to the server, while it concurrently accepts characters that the server sends back and displays them on the user's screen. The server must accept a TCP connection from the client, and then relay data between the TCP connection and the local operating system.

In practice, the server is more complex than the figure shows because it must handle multiple, concurrent connections. Usually, a master server process waits for new connections and creates a new slave to handle each connection. Thus, the 'TELNET server', shown in Figure 25.1, represents the slave that handles one particular connection. The figure does not show the master server that listens for new requests, nor does it show the slaves handling other connections.

We use the term *pseudo terminal*[†] to describe the operating system entry point that allows a running program like the TELNET server to transfer characters to the operating system as if they came from a keyboard. It is impossible to build a TELNET server unless the operating system supplies such a facility. If the system supports a pseudo terminal abstraction, the TELNET server can be implemented with application programs. Each slave server connects a TCP stream from one client to a particular pseudo terminal.

Arranging for the TELNET server to be an application level program has advantages and disadvantages. The most obvious advantage is that it makes modification and control of the server easier than if the code were embedded in the operating system. The obvious disadvantage is inefficiency. Each keystroke travels from the user's keyboard through the operating system to the client program, from the client program back through the operating system and across the internet to the server machine. After reaching the destination machine, the data must travel up through the server's operating system to the server application program, and from the server application program back into the server's operating system at a pseudo terminal entry point. Finally, the remote operating system delivers the character to the application program the user is running. Meanwhile, output (including remote character echo if that option has been selected) travels back from the server to the client over the same path.

Readers who understand operating systems will appreciate that for the implementation shown in Figure 25.1, every keystroke requires computers to switch process context several times. In most systems, an additional context switch is required because the operating system on the server's machine must pass characters from the pseudo terminal back to another application program (e.g., a command interpreter). Although context switching is expensive, the scheme is practical because users do not type at high speed.

25.4 Accommodating Heterogeneity

To make TELNET interoperate between as many systems as possible, it must accommodate the details of heterogeneous computers and operating systems. For example, some systems require lines of text to be terminated by the ASCII *carriage control* character (CR). Others require the ASCII *linefeed* (LF) character. Still others require the two-character sequence of CR-LF. In addition, most interactive systems provide a way for a user to enter a key that interrupts a running program. However, the specific keystroke used to interrupt a program varies from system to system (e.g., some systems use Control-C, while others use ESCAPE).

To accommodate heterogeneity, TELNET defines how data and command sequences are sent across the Internet. The definition is known as the *network virtual terminal* (NVT). As Figure 25.2 illustrates, the client software translates keystrokes and command sequences from the user's terminal into NVT format and sends them to the server. Server software translates incoming data and commands from NVT format into the format the remote system requires. For data returning, the remote server translates from the remote machine's format to NVT, and the local client translates from NVT to the local machine's format.

[†]UNIX calls the system entry point a *pseudo tty* because character-oriented devices are called *ttys*.

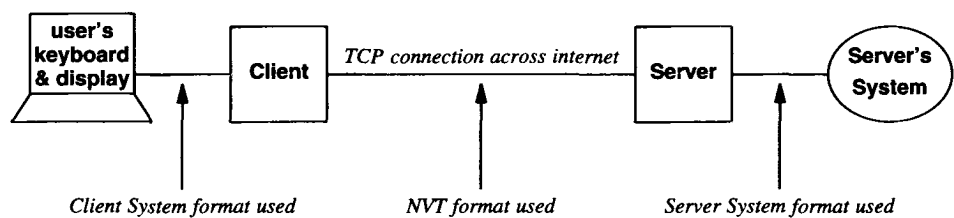


Figure 25.2 Use of the Network Virtual Terminal (NVT) format by TELNET.

The definition of NVT format is fairly straightforward. All communication involves 8-bit bytes. At startup, NVT uses the standard 7-bit USASCII representation for data and reserves bytes with the high order bit set for command sequences. The US-ASCII character set includes 95 characters that have “printable” graphics (e.g., letters, digits, and punctuation marks) as well as 33 “control” codes. All printable characters are assigned the same meaning as in the standard USASCII character set. The NVT standard defines interpretations for control characters as shown in Figure 25.3†.

ASCII Control Code	Decimal Value	Assigned Meaning
NUL	0	No operation (has no effect on output)
BEL	7	Sound audible/visible signal (no motion)
BS	8	Move left one character position
HT	9	Move right to the next horizontal tab stop
LF	10	Move down (vertically) to the next line
VT	11	Move down to the next vertical tab stop
FF	12	Move to the top of the next page
CR	13	Move to the left margin on the current line
other control	—	No operation (has no effect on output)

Figure 25.3 The TELNET NVT interpretation of USASCII control characters.
TELNET does not specify the locations of tab stops.

In addition to the control character interpretation in Figure 25.3, NVT defines the standard line termination to be a two-character sequence *CR-LF*. When a user presses the key that corresponds to end-of-line on the local terminal (e.g., *ENTER* or *RETURN*), the TELNET client must map it into *CR-LF* for transmission. The TELNET server translates *CR-LF* into the appropriate end-of-line character sequence for the remote machine.

†The NVT interpretation of control characters follows the usual ASCII interpretation.

25.5 Passing Commands That Control The Remote Side

We said that most systems provide a mechanism that allows users to terminate a running program. Usually, the local operating system binds such mechanisms to a particular key or keystroke sequence. For example, unless the user specifies otherwise, many UNIX systems reserve the character generated by *CONTROL-C* as the interrupt key. Depressing *CONTROL-C* causes UNIX to terminate the executing program; the program does not receive *CONTROL-C* as input. The system may reserve other characters or character sequences for other control functions.

TELNET NVT accommodates control functions by defining how they are passed from the client to the server. Conceptually, we think of NVT as accepting input from a keyboard that can generate more than 128 possible characters. We assume the user’s keyboard has virtual (imaginary) keys that correspond to the functions typically used to control processing. For example, NVT defines a conceptual “interrupt” key that requests program termination. Figure 25.4 lists the control functions that NVT allows.

Signal	Meaning
IP	Interrupt Process (terminate running program)
AO	Abort Output (discard any buffered output)
AYT	Are You There (test if server is responding)
EC	Erase Character (delete the previous character)
EL	Erase Line (delete the entire current line)
SYNCH	Synchronize (clear data path until TCP urgent data point, but do interpret commands)
BRK	Break (break key or attention signal)

Figure 25.4 The control functions TELNET NVT recognizes. Conceptually, the client receives these from a user in addition to normal data, and passes them to the server’s system where they must be interpreted.

In practice, most keyboards do not provide extra keys for commands. Instead, individual operating systems or command interpreters have a variety of ways to generate them. We already mentioned the most common technique: binding an individual ASCII character to a control function so when the user presses the key, the operating system takes the appropriate action instead of accepting the character as input. The NVT designers chose to keep commands separate from the normal ASCII character set for two reasons. First, defining the control functions separately means TELNET has greater flexibility. It can transfer all possible ASCII character sequences between client and server as well as all possible control functions. Second, by separating signals from normal data, NVT allows the client to specify signals unambiguously — there is never confusion about whether an input character should be treated as data or as a control function.

To pass control functions across the TCP connection, TELNET encodes them using an *escape sequence*. An escape sequence uses a reserved octet to indicate that a control code octet follows. In TELNET, the reserved octet that starts an escape sequence is known as the *interpret as command (IAC)* octet. Figure 25.5 lists the possible commands and the decimal encoding used for each.

Command	Decimal Encoding	Meaning
IAC	255	Interpret next octet as command (when the IAC octet appears as data, the sender doubles it and sends the 2-octet sequence IAC-IAC)
DON'T	254	Denial of request to perform specified option
DO	253	Approval to allow specified option
WON'T	252	Refusal to perform specified option
WILL	251	Agreement to perform specified option
SB	250	Start of option subnegotiation
GA	249	The "go ahead" signal
EL	248	The "erase line" signal
EC	247	The "erase character" signal
AYT	246	The "are you there" signal
AO	245	The "abort output" signal
IP	244	The "interrupt process" signal
BRK	243	The "break" signal
DMARK	242	The data stream portion of a SYNCH (always accompanied by TCP Urgent notification)
NOP	241	No operation
SE	240	End of option subnegotiation
EOR	239	End of record

Figure 25.5 TELNET commands and encoding for each. The codes only have meaning if preceded by an *IAC* character. When *IAC* occurs in the data, it is sent twice.

As the figure shows, the signals generated by conceptual keys on an NVT keyboard each have a corresponding command. For example, to request that the server interrupt the executing program, the client must send the 2-octet sequence *IAC IP* (255 followed by 244). Additional commands allow the client and server to negotiate which options they will use and to synchronize communication.

25.6 Forcing The Server To Read A Control Function

Sending control functions along with normal data is not always sufficient to guarantee the desired results. To see why, consider the situation under which a user might send the *interrupt process* control function to the server. Usually, such control is only needed when the program executing on the remote machine is misbehaving and the user wants the server to terminate the program. For example, the program might be executing an endless loop without reading input or generating output. Unfortunately, if the application at the server's site stops reading input, operating system buffers will eventually fill and the server will be unable to write more data to the pseudo terminal. When this happens, the server must stop reading data from the TCP connection, causing its buffers to fill. Eventually, TCP on the server machine will begin advertising a zero window size, preventing data from flowing across the connection.

If the user generates an interrupt control function when buffers are filled, the control function will never reach the server. That is, the client can form the command sequence *IAC IP* and write it to the TCP connection, but because TCP has stopped sending to the server's machine, the server will not read the control sequence. The point is:

TELNET cannot rely on the conventional data stream alone to carry control sequences between client and server, because a misbehaving application that needs to be controlled might inadvertently block the data stream.

To solve the problem, TELNET uses an *out of band* signal. TCP implements out of band signaling with the *urgent data* mechanism. Whenever it places a control function in the data stream, TELNET also sends a *SYNCH* command. TELNET then appends a reserved octet called the *data mark*, and causes TCP to signal the server by sending a segment with the URGENT DATA bit set. Segments carrying urgent data bypass flow control and reach the server immediately. In response to an urgent signal, the server reads and discards all data until it finds the data mark. The server returns to normal processing when it encounters the data mark.

25.7 TELNET Options

Our simple description of TELNET omits one of the most complex aspects: options. In TELNET, options are negotiable, making it possible for the client and server to reconfigure their connection. For example, we said that usually the data stream passes 7-bit data and uses octets with the eighth bit set to pass control information like the *Interrupt Process* command. However, TELNET also provides an option that allows the client and server to pass 8-bit data (when passing 8-bit data, the reserved octet *IAC* must still be doubled if it appears in the data). The client and server must negotiate, and both must agree to pass 8-bit data before such transfers are possible.

The range of TELNET options is wide: some extend the capabilities in major ways while others deal with minor details. For example, the original protocol was designed for a half-duplex environment where it was necessary to tell the other end to “go ahead” before it would send more data. One of the options controls whether TELNET operates in half- or full-duplex mode. Another option allows the server on a remote machine to determine the user’s terminal type. The terminal type is important for software that generates cursor positioning sequences (e.g., a full screen editor executing on a remote machine).

Figure 25.6 lists several of the most commonly implemented TELNET options.

Name	Code	RFC	Meaning
Transmit Binary	0	856	Change transmission to 8-bit binary
Echo	1	857	Allow one side to echo data it receives
Suppress-GA	3	858	Suppress (no longer send) Go-ahead signal after data
Status	5	859	Request for status of a TELNET option from remote site
Timing-Mark	6	860	Request timing mark be inserted in return stream to synchronize two ends of a connection
Terminal-Type	24	884	Exchange information about the make and model of a terminal being used (allows programs to tailor output like cursor positioning sequences for the user’s terminal)
End-of-Record	25	885	Terminate data sent with EOR code
Linemode	34	1116	Use local editing and send complete lines instead of individual characters

Figure 25.6 Commonly used TELNET options.

25.8 TELNET Option Negotiation

The way TELNET negotiates options is interesting. Because it sometimes makes sense for the server to initiate a particular option, the protocol is designed to allow either end to make a request. Thus, the protocol is said to be *symmetric* with respect to option processing. The receiving end either responds to a request with a positive acceptance or a rejection. In TELNET terminology, the request is *WILL X*, meaning *will you agree to let me use option X*; and the response is either *DO X* or *DON'T X*, meaning *I do agree to let you use option X* or *I don't agree to let you use option X*. The symmetry arises because *DO X* requests that the receiving party begin using option X, and *WILL X* or *WON'T X* means *I will start using option X* or *I won't start using it*[†].

[†]To eliminate potential loops that arise when two sides each think the other’s acknowledgement is a request, the protocol specifies that no acknowledgement be given to a request for an option that is already in use.

Another interesting negotiation concept arises because both ends are required to run an unenhanced NVT implementation (i.e., one without any options turned on). If one side tries to negotiate an option that the other does not understand, the side receiving the request can simply decline. Thus, it is possible to interoperate newer, more sophisticated versions of TELNET clients and servers (i.e., software that understands more options) with older, less sophisticated versions. If both the client and server understand the new options, they may be able to improve interaction. If not, they will revert to a less efficient, but workable style.

We can summarize:

TELNET uses a symmetric option negotiation mechanism to allow clients and servers to reconfigure the parameters controlling their interaction. Because all TELNET software understands a basic NVT protocol, clients and servers can interoperate even if one understands options another does not.

25.9 Rlogin (BSD UNIX)

Operating systems derived from BSD UNIX include a remote login service, *rlogin*, that supports trusted hosts. It allows system administrators to choose a set of machines over which login names and file access protections are shared and to establish equivalences among user logins. Users can control access to their accounts by authorizing remote login based on remote host and remote user name. Thus, it is possible for a user to have login name *X* on one machine and *Y* on another, and still be able to remotely login from one of the machines to the other without typing a password each time.

Having automatic authorization makes remote login facilities useful for general purpose programs as well as human interaction. One variant of the *rlogin* command, *rsh*, invokes a command interpreter on the remote UNIX machine and passes the command line arguments to the command interpreter, skipping the login step completely. The format of a command invocation using *rsh* is:

`rsh machine command`

Thus, typing

`rsh merlin ps`

on any of the machines in the Computer Science Department at Purdue University executes the *ps* command on machine *merlin*, with UNIX's standard input and standard output connected across the network to the user's keyboard and display. The user sees the output as if he or she were logged into machine *merlin*. Because the user can arrange to have *rsh* invoke remote commands without prompting for a password, it can be used in programs as well as from the keyboard.

Because protocols like *rlogin* understand both the local and remote computing environments, they communicate better than general purpose remote login protocols like TELNET. For example, *rlogin* understands the UNIX notions of *standard input*, *standard output*, and *standard error*, and uses TCP to connect them to the remote machine. Thus, it is possible to type

```
rsh merlin ps > filename
```

and have output from the remote command redirected† into file *filename*. *Rlogin* also understands terminal control functions like flow control characters (typically Control-S and Control-Q). It arranges to stop output immediately without waiting for the delay required to send them across the network to the remote host. Finally, *rlogin* exports part of the user's environment to the remote machine, including information like the user's terminal type (i.e., the *TERM* variable). As a result, a remote login session appears to behave almost exactly like a local login session.

25.10 Summary

Much of the rich functionality associated with TCP/IP results from a variety of high-level services supplied by application programs. The high-level remote login protocols these programs use build on the basic services: unreliable datagram delivery and reliable stream transport. The services usually follow the client-server model in which servers operate at known protocol ports so clients know how to contact them.

We reviewed two remote login systems: TELNET, the TCP/IP internet standard, and *rlogin*, a popular protocol used with systems derived from BSD UNIX. TELNET provides a basic service. It allows the client to pass commands such as *interrupt process* as well as data to the server. It also permits a client and server to negotiate many options. In contrast to TELNET, *rlogin* allows system managers and users more flexibility in establishing the equivalence of accounts on multiple machines, but it is not as widely available as TELNET.

FOR FURTHER STUDY

Many high-level protocols have been proposed, but only a few are in common use. Edge [1979] compares end-to-end protocols with the hop-by-hop approach. Saltzer, Reed, and Clark [1984] argues for having the highest level protocols perform end-to-end acknowledgement and error detection.

Postel [RFC 854] contains the TELNET remote login protocol specification. It was preceded by over three dozen RFCs that discuss TELNET options, weaknesses, experiments, and proposed changes, including Postel [RFC 764] that contains an earlier standard. Postel and Reynolds [RFC 855] gives a specification for options and consid-

†The “greater than” symbol is the usual UNIX syntax for directing the output of a command into a file.

ers subnegotiation. A lengthy list of options can be found in RFCs 856, 857, 858, 859, 860, 861, 884, 885, 1041, 1091, 1096, 1097, 1184, 1372, 1416, and 1572. The program *tn3270* uses a TELNET-like mechanism to provide access to IBM computers running the VM/CMS operating system [RFCs 1576, 1646 and 1647]; Rekhter [RFC 1041] covers the TELNET option that permits communication with IBM 3270 displays.

EXERCISES

- 25.1 Experiment with both TELNET and *rlogin*. What are the noticeable differences?
- 25.2 Despite the large volume of notes written about TELNET, it can be argued that the protocol is still not well-defined. Experiment with TELNET: use it to reach a machine, *A*, and invoke TELNET on *A* to reach a second machine, *B*. Does the combination of two TELNET connections handle *line feed* and *carriage control* characters properly?
- 25.3 What is a remote procedure call?
- 25.4 Folklore says that operating systems come and go while protocols last forever. Test this axiom by surveying your local computing site to see whether operating systems or communication protocols have changed more frequently.
- 25.5 Build TELNET client software.
- 25.6 Use a TELNET client to connect your keyboard and display to the TCP protocol port for *echo* or *chargen* on your local system to see what happens.
- 25.7 Read the TELNET standard and find out how the SYNCH operation works.
- 25.8 TELNET uses TCP's *urgent data* mechanism to force the remote operating system to respond to control functions quickly. Read the standard to find out which commands the remote server honors while scanning the input stream.
- 25.9 How can the symmetric DO/DON'T — WILL/WON'T option negotiation produce an endless loop of responses if the other party *always* acknowledges a request?
- 25.10 RFC 854 (the TELNET protocol specification) contains exactly 854 lines of text. Do you think there is cosmic significance in this?

26

Applications: File Transfer And Access (FTP, TFTP, NFS)

26.1 Introduction

This chapter continues our exploration of application protocols. It examines the file access and transfer protocols that are part of the TCP/IP protocol suite. It describes their design and shows an example of a typical user interface. We will learn that the most widely used file transfer protocol builds on TCP, covered in Chapter 13, and TELNET, described in the previous chapter.

26.2 File Access And Transfer

Many network systems provide computers with the ability to access files on remote machines. Designers have explored a variety of approaches to remote access; each approach optimizes for a particular set of goals. For example, some designs use remote file access to lower overall cost. In such architectures, a single, centralized *file server* provides secondary storage for a set of inexpensive computers that have no local disk storage. For example, the diskless machines can be portable, hand-held devices used for chores such as inventory. Such machines communicate with a file server over a high-speed wireless network.

Some designs use remote storage to archive data. In such designs, users have conventional computers with local storage facilities and operate them as usual. Periodically the conventional computers send copies of files (or copies of entire disks) across a network to an archival facility, where they are stored in case of accidental loss.

Finally, some designs emphasize the ability to share data across multiple programs, multiple users, or multiple sites. For example, an organization might choose to have a single on-line database of outstanding orders shared by all groups in the organization.

26.3 On-line Shared Access

File sharing comes in two distinct forms: *on-line access* and *whole-file copying*. Shared on-line access means allowing multiple programs to access a single file concurrently. Changes to the file take effect immediately and are available to all programs that access the file. Whole-file copying means that whenever a program wants to access a file, it obtains a local copy. Copying is often used for read-only data, but if the file must be modified, the program makes changes to the local copy and transfers a copy of the modified file back to the original site.

Many users think that on-line data sharing can only be provided by a database system that operates as a server and allows users (clients) to contact it from remote sites. However, file sharing is usually more sophisticated and easier to use. For example, a file system that provides shared, on-line access for remote users does not necessarily require a user to invoke a special client program as a database system does. Instead, the operating system provides access to remote, shared files exactly the same way it provides access to local files. A user can execute any application program using a remote file as input or output. We say that the remote file is *integrated* with local files, and that the entire file system provides *transparent access* to shared files.

The advantage of transparent access should be obvious: remote file access occurs with no visible changes to application programs. Users can access both local and remote files, allowing them to perform arbitrary computations on shared data. The disadvantages are less obvious. Users may be surprised by the results. For example, consider an application program that uses both local and remote files. If the network or the remote machine is down, the application program may not work even though the user's machine is operating. Even if the remote machine is operating, it may be overloaded or the network may be congested, causing the application program to run slowly, or causing communication protocols to report timeout conditions that the user does not expect. The application program seems unreliable.

Despite its advantages, implementing integrated, transparent file access can be difficult. In a heterogeneous environment, file names available on one computer may be impossible to map into the file namespace of another. Similarly, a remote file access mechanism must handle notions of ownership, authorization, and access protection, which do not transcend computer system boundaries. Finally, because file representations and allowed operations vary from machine to machine, it may be difficult or impossible to implement all operations on all files.

26.4 Sharing By File Transfer

The alternative to integrated, transparent on-line access is *file transfer*. Accessing remote data with a transfer mechanism is a two-step process: the user first obtains a local copy of a file and then operates on the copy. Most transfer mechanisms operate outside the local file system (i.e., they are not integrated). A user must invoke a special-purpose client program to transfer files. When invoking the client, the user specifies a remote computer on which the desired file resides and, possibly, an authorization needed to obtain access (e.g., an account or password). The client contacts a server on the remote machine and requests a copy of the file. Once the transfer is complete, the user terminates the client and uses application programs on the local system to read or modify the local copy. One advantage of whole-file copying lies in the efficiency of operations — once a program has obtained a copy of a remote file, it can manipulate the copy efficiently. Thus, many computations run faster with whole-file copying than with remote file access.

As with on-line sharing, whole-file transfer between heterogeneous machines can be difficult. The client and server must agree on authorization, notions of file ownership and access protections, and data formats. The latter is especially important because it may make inverse translations impossible. To see why, consider copying between two machines, *A* and *B*, that use different representations for floating point numbers as well as different representations for text files. As most programmers realize, it may be impossible to convert from one machine's floating point format to another's without losing precision. The same can happen with text files. Suppose system *A* stores text files as variable-length lines and system *B* pads text lines to a fixed length. Transferring a file from *A* to *B* and back can add padding to every line, making the final copy different from the original. However, automatically removing padding from the ends of lines during the transfer back to *A* will also make the copy different from the original for any files that had padding on some lines.

The exact details of differences in representation and the techniques to handle them depend on the computer systems involved. Furthermore, we have seen that not all representational differences can be accommodated — information can be lost when data must be translated from one representation to another. While it is not essential to learn about all possible representational differences, remembering that TCP/IP is designed for a heterogeneous environment will help explain some of the features of the TCP/IP file transfer protocols.

26.5 FTP: The Major TCP/IP File Transfer Protocol

File transfer is among the most frequently used TCP/IP applications, and it accounts for much network traffic. Standard file transfer protocols existed for the ARPANET before TCP/IP became operational. These early versions of file transfer software evolved into a current standard known as the *File Transfer Protocol (FTP)*.

26.6 FTP Features

Given a reliable end-to-end transport protocol like TCP, file transfer might seem trivial. However, as the previous sections pointed out, the details of authorization, naming, and representation among heterogeneous machines make the protocol complex. In addition, FTP offers many facilities beyond the transfer function itself.

- *Interactive Access.* Although FTP is designed to be used by programs, most implementations provide an interactive interface that allows humans to easily interact with remote servers. For example, a user can ask for a listing of all files in a directory on a remote machine. Also, the client usually responds to the input “help” by showing the user information about possible commands that can be invoked.

- *Format (representation) Specification.* FTP allows the client to specify the type and format of stored data. For example, the user can specify whether a file contains text or binary integers and whether text files use the ASCII or EBCDIC character sets.

- *Authentication Control.* FTP requires clients to authorize themselves by sending a login name and password to the server before requesting file transfers. The server refuses access to clients that cannot supply a valid login and password.

26.7 FTP Process Model

Like other servers, most FTP server implementations allow concurrent access by multiple clients. Clients use TCP to connect to a server. As described in Chapter 21, a single master server process awaits connections and creates a slave process to handle each connection. Unlike most servers, however, the slave process does not perform all the necessary computation. Instead, the slave accepts and handles the *control connection* from the client, but uses an additional process or processes to handle a separate *data transfer connection*. The control connection carries commands that tell the server which file to transfer. The data transfer connection, which also uses TCP as the transport protocol, carries all data transfers.

Usually, both the client and server create a separate process to handle the data transfer. While the exact details of the process architecture depend on the operating systems used, Figure 26.1 illustrates the concept:

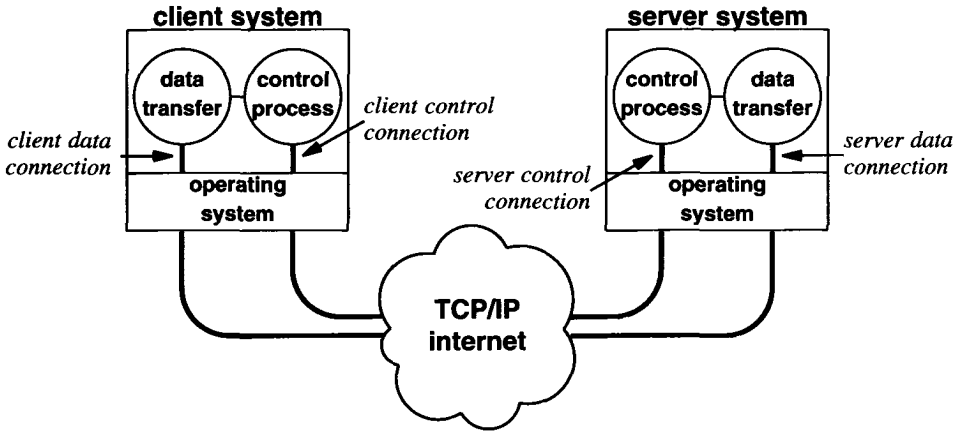


Figure 26.1 An FTP client and server with a TCP control connection between them and a separate TCP connection between their associated data transfer processes.

As the figure shows, the client control process connects to the server control process using one TCP connection, while the associated data transfer processes use their own TCP connection. In general, the control processes and the control connection remain alive as long as the user keeps the FTP session active. However, FTP establishes a new data transfer connection for each file transfer. In fact, many implementations create a new pair of data transfer processes, as well as a new TCP connection, whenever the server needs to send information to the client. The idea can be summarized:

Data transfer connections and the data transfer processes that use them can be created dynamically when needed, but the control connection persists throughout a session. Once the control connection disappears, the session is terminated and the software at both ends terminates all data transfer processes.

Of course, client implementations that execute on a computer without operating system support for multiple processes may have a less complex structure. Such implementations often sacrifice generality by using a single application program to perform both the data transfer and control functions. However, the protocol requires that such clients still use multiple TCP connections, one for control and the other(s) for data transfer.

26.8 TCP Port Number Assignment

When a client forms an initial connection to a server, the client uses a random, locally assigned, protocol port number, but contacts the server at a well-known port (21). As Chapter 21 points out, a server that uses only one protocol port can accept connections from many clients because TCP uses both endpoints to identify a connection. The question arises, “When the control processes create a new TCP connection for a given data transfer, what protocol port numbers do they use?” Obviously, they cannot use the same pair of port numbers used in the control connection. Instead, the client obtains an unused port on its machine, which will be used for a TCP connection with the data transfer process on the server’s machine. The data transfer process on the server machine uses the well-known port reserved for FTP data transfer (20). To ensure that a data transfer process on the server connects to the correct data transfer process on the client machine, the server side must not accept connections from an arbitrary process. Instead, when it issues the TCP active open request, a server specifies the port that will be used on the client machine as well as the local port.

We can see why the protocol uses two connections — the client control process obtains a local port to be used in the file transfer, creates a transfer process on the client machine to listen at that port, communicates the port number to the server over the control connection, and then waits for the server to establish a TCP connection to the port. In general:

In addition to passing user commands to the server, FTP uses the control connection to allow client and server control processes to coordinate their use of dynamically assigned TCP protocol ports and the creation of data transfer processes that use those ports.

What format should FTP use for data passing across the control connection? Although they could have invented a new specification, the designers of FTP did not. Instead, they allow FTP to use the TELNET network virtual terminal protocol described in Chapter 25. Unlike the full TELNET protocol, FTP does not allow option negotiation; it uses only the basic NVT definition. Thus, management of an FTP control connection is much simpler than management of a standard TELNET connection. Despite its limitations, using the TELNET definition instead of inventing a new one helps simplify FTP considerably.

26.9 The User’s View Of FTP

Users view FTP as an interactive system. Once invoked, the client performs the following operations repeatedly: read a line of input, parse the line to extract a command and its arguments, and execute the command with the specified arguments. For example, to initiate the version of FTP available under UNIX, the user invokes the *ftp* command:

% ftp

The local FTP client program begins and issues a prompt to the user. Following the prompt, the user can issue commands like *help*.

```
ftp> help
```

Commands may be abbreviated. Commands are:

!	cr	macdef	proxy	send
\$	delete	mdelete	sendport	status
account	debug	mdir	put	struct
append	dir	mget	pwd	sunique
ascii	disconnect	mkdir	quit	tenex
bell	form	mls	quote	trace
binary	get	mode	recv	type
bye	glob	mput	remotehelp	user
case	hash	nmap	rename	verbose
cd	help	ntrans	reset	?
cdup	lcd	open	rmdir	
close	ls	prompt	runique	

To obtain more information about a given command the user types *help command* as in the following examples (output is shown in the format *ftp* produces):

```
ftp> help ls
```

```
ls                list contents of remote directory
```

```
ftp> help cdup
```

```
cdup              change remote working directory to parent directory
```

```
ftp> help glob
```

```
glob              toggle metacharacter expansion of local file names
```

```
ftp> help bell
```

```
bell              beep when command completed
```

To execute a command, the user types the command name:

```
ftp> bell
```

```
Bell mode on.
```

26.10 An Example Anonymous FTP Session

While the access authorization facilities in FTP make it more secure, strict enforcement prohibits an arbitrary client from accessing any file until they obtain a login and password for the computer on which the server operates. To provide access to public files, many TCP/IP sites allow *anonymous FTP*. Anonymous FTP access means a client does not need an account or password. Instead, the user specifies login name *anonymous* and password *guest*. The server allows anonymous logins, but restricts access to only publicly available files†.

Usually, users execute only a few FTP commands to establish a connection and obtain a file; few users have ever tried most commands. For example, suppose someone has placed an on-line copy of a text in file *tcpbook.tar* in the subdirectory *pub/comer* on machine *ftp.cs.purdue.edu*. A user logged in at another site as *usera* could obtain a copy of the file by executing the following:

```
% ftp ftp.cs.purdue.edu
Connected to lucan.cs.purdue.edu.
220 lucan.cs.purdue.edu FTP server (Version wu-2.4.2-VR16(1) ready.
Name (ftp.cs.purdue.edu:usera): anonymous
331 Guest login ok, send e-mail address as password.
Password: guest
230 Guest login ok, access restrictions apply.
ftp> get pub/comer/tcpbook.tar bookfile
200 PORT command okay.
150 Opening ASCII mode data connection for tcpbook.tar (9895469 bytes).
226 Transfer complete.
9895469 bytes received in 22.76 seconds (4.3e+02 Kbytes/s)
ftp> close
221 Goodbye.
ftp> quit
```

In this example, the user specifies machine *ftp.cs.purdue.edu* as an argument to the FTP command, so the client automatically opens a connection and prompts for authorization. The user invokes anonymous FTP by specifying login *anonymous* and password *guest*‡ (although our example shows the password that the user types, the ftp program does not display it on the user's screen).

After typing a login and password, the user requests a copy of a file using the *get* command. In the example, the *get* command is followed by two arguments that specify the remote file name and a name for the local copy. The remote file name is *pub/comer/tcpbook.tar* and the local copy will be placed in *bookfile*. Once the transfer completes, the user types *close* to break the connection with the server, and types *quit* to leave the client.

†In many UNIX systems, the server restricts anonymous FTP by changing the file system root to a small, restricted directory (e.g., */usr/ftp*).

‡In practice, the server emits additional messages that request the user to use an e-mail address instead of *guest*.

Intermingled with the commands the user types are informational messages. FTP messages always begin with a 3-digit number followed by text. Most come from the server; other output comes from the local client. For example, the message that begins 220 comes from the server and contains the domain name of the machine on which the server executes. The statistics that report the number of bytes received and the rate of transfer come from the client. In general:

Control and error messages between the FTP client and server begin with a 3-digit number followed by text. The software interprets the number; the text is meant for humans.

The example session also illustrates a feature of FTP described earlier: the creation of new TCP connections for data transfer. Notice the *PORT* command in the output. The client *PORT* command reports that a new TCP port number has been obtained for use as a data connection. The client sends the port information to the server over the control connection; data transfer processes at both ends use the new port number when forming a connection. After the transfer completes, the data transfer processes at each end close the connection.

26.11 TFTP

Although FTP is the most general file transfer protocol in the TCP/IP suite, it is also the most complex and difficult to program. Many applications do not need the full functionality FTP offers, nor can they afford the complexity. For example, FTP requires clients and servers to manage multiple concurrent TCP connections, something that may be difficult or impossible on personal computers that do not have sophisticated operating systems.

The TCP/IP suite contains a second file transfer protocol that provides inexpensive, unsophisticated service. Known as the *Trivial File Transfer Protocol*, or (*TFTP*), it is intended for applications that do not need complex interactions between the client and server. TFTP restricts operations to simple file transfers and does not provide authentication. Because it is more restrictive, TFTP software is much smaller than FTP.

Small size is important in many applications. For example, manufacturers of diskless devices can encode TFTP in read-only memory (ROM) and use it to obtain an initial memory image when the machine is powered on. The program in ROM is called the system *bootstrap*[†]. The advantage of using TFTP is that it allows bootstrapping code to use the same underlying TCP/IP protocols that the operating system uses once it begins execution. Thus, it is possible for a computer to bootstrap from a server on another physical network.

Unlike FTP, TFTP does not need a reliable stream transport service. It runs on top of UDP or any other unreliable packet delivery system, using timeout and retransmission to ensure that data arrives. The sending side transmits a file in fixed size (512 byte) blocks and awaits an acknowledgement for each block before sending the next. The receiver acknowledges each block upon receipt.

[†]Chapter 23 discusses the details of bootstrapping with DHCP.

The rules for TFTP are simple. The first packet sent requests a file transfer and establishes the interaction between client and server — the packet specifies a file name and whether the file will be read (transferred to the client) or written (transferred to the server). Blocks of the file are numbered consecutively starting at 1. Each data packet contains a header that specifies the number of the block it carries, and each acknowledgement contains the number of the block being acknowledged. A block of less than 512 bytes signals the end of file. It is possible to send an error message either in the place of data or an acknowledgement; errors terminate the transfer.

Figure 26.2 shows the format of the five TFTP packet types. The initial packet must use operation codes 1 or 2, specifying either a *read request* or a *write request*. The initial packet contains the name of the file as well as the access mode the client requests (*read* access or *write* access).

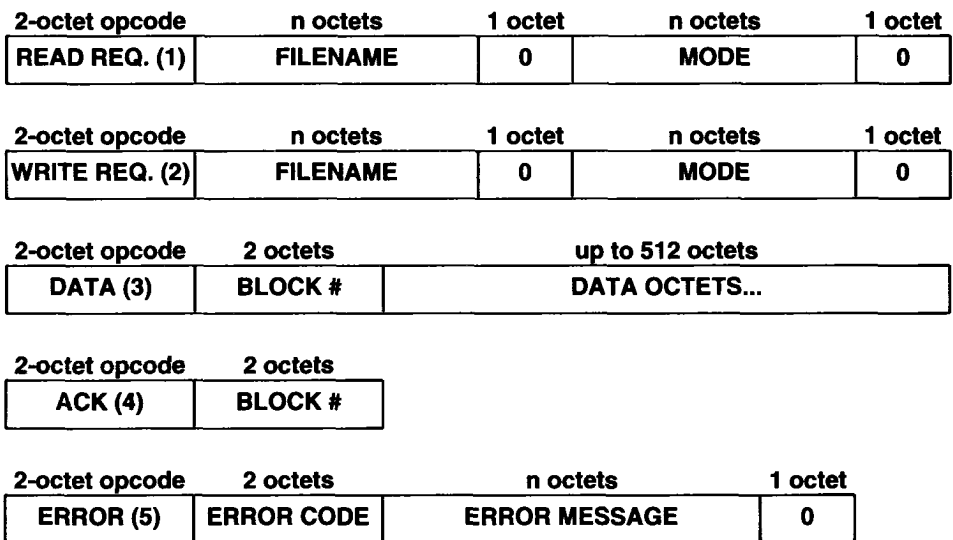


Figure 26.2 The five TFTP message types. Fields are not shown to scale because some are variable length; an initial 2-octet operation code identifies the message format.

Once a *read* or *write* request has been made, the server uses the IP address and UDP protocol port number of the client to identify subsequent operations. Thus, neither *data* messages (the messages that carry blocks from the file) nor *ack* messages (the messages that acknowledge data blocks) need to specify the file name. The final message type illustrated in Figure 26.2 is used to report errors. Lost messages can be retransmitted after a timeout, but most other errors simply cause termination of the interaction.

TFTP retransmission is unusual because it is symmetric. Each side implements a timeout and retransmission. If the side sending data times out, it retransmits the last data block. If the side responsible for acknowledgements times out, it retransmits the last acknowledgement. Having both sides participate in retransmission helps ensure that transfer will not fail after a single packet loss.

While symmetric retransmission guarantees robustness, it can lead to excessive retransmissions. The problem, known as the *Sorcerer's Apprentice Bug*, arises when an acknowledgement for data packet k is delayed, but not lost. The sender retransmits the data packet, which the receiver acknowledges. Both acknowledgements eventually arrive, and each triggers a transmission of data packet $k+1$. The receiver will acknowledge both copies of data packet $k+1$, and the two acknowledgements will each cause the sender to transmit data packet $k+2$. The Sorcerer's Apprentice Bug can also start if the underlying internet duplicates packets. Once started, the cycle continues indefinitely with each data packet being transmitted exactly twice.

Although TFTP contains little except the minimum needed for transfer, it does support multiple file types. One interesting aspect of TFTP allows it to be integrated with electronic mail[†]. A client can specify to the server that it will send a file that should be treated as mail with the *FILENAME* field taken to be the name of a mailbox to which the server should deliver the message.

26.12 NFS

Initially developed by Sun Microsystems Incorporated, the *Network File System (NFS)* provides on-line shared file access that is transparent and integrated; many TCP/IP sites use NFS to interconnect their computers' file systems. From the user's perspective, NFS is almost invisible. A user can execute an arbitrary application program and use arbitrary files for input or output. The file names themselves do not show whether the files are local or remote.

26.13 NFS Implementation

Figure 26.3 illustrates how NFS is embedded in an operating system. When an application program executes, it calls the operating system to *open* a file, or to *store* and *retrieve* data in files. The file access mechanism accepts the request and automatically passes it to either the local file system software or to the NFS client, depending on whether the file is on the local disk or on a remote machine. When it receives a request, the client software uses the NFS protocol to contact the appropriate server on a remote machine and perform the requested operation. When the remote server replies, the client software returns the results to the application program.

[†]In practice, the use of TFTP as a mail transport is discouraged. Refer to Chapter 27 for details on electronic mail.

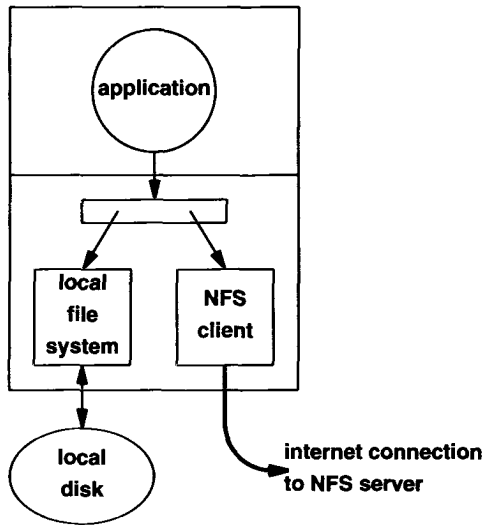


Figure 26.3 NFS code in an operating system. When an application program requests a file operation, the operating system must pass the request to the local file system or to the NFS client software.

26.14 Remote Procedure Call (RPC)

Instead of defining the NFS protocol from scratch, the designers chose to build three independent pieces: the NFS protocol itself, a general-purpose *Remote Procedure Call (RPC)* mechanism, and a general-purpose *eXternal Data Representation (XDR)*. Their intent was to separate the three to make it possible to use RPC and XDR in other software, including application programs as well as other protocols.

From the programmer's point of view, NFS itself provides no new procedures that a program can call. Instead, once a manager has configured NFS, programs access remote files using exactly the same operations as they use for local files. However, both RPC and XDR provide mechanisms that programmers can use to build distributed programs. For example, a programmer can divide a program into a client side and a server side that use RPC as the chief communication mechanism. On the client side, the programmer designates some procedures as *remote*, forcing the compiler to incorporate RPC code into those procedures. On the server side, the programmer implements the desired procedures and uses other RPC facilities to declare them to be part of a server. When the executing client program calls one of the remote procedures, RPC automatically collects values for arguments, forms a message, sends the message to the remote server, awaits a response, and stores returned values in the designated arguments. In essence, communication with the remote server occurs automatically as a side-effect of a remote procedure call. The RPC mechanism hides all the details of protocols, making it possible for programmers who know little about the underlying communication protocols to write distributed programs.